ENHANCING THE SECURITY OF CRYPTOGRAPHIC IMPLEMENTATIONS

by

Yongming Fan

A Dissertation

Submitted to the Faculty of Purdue University

In Partial Fulfillment of the Requirements for the degree of

Doctor of Philosophy

Department of Computer Science

West Lafayette, Indiana

August 2026

THE PURDUE UNIVERSITY GRADUATE SCHOOL
STATEMENT OF COMMITTEE APPROVAL

Dr. Christina Garman, Chair

Department of Computer Science

Dr. Jeremiah Blocki

Department of Computer Science

Dr. Dave (Jing) Tian

Department of Computer Science

Dr. Antonio Bianchi

Department of Computer Science

Approved by:

Dr. Petros Drineas

To graduate students

ACKNOWLEDGMENTS

Purdue University's Engineering Computer Network (now part of Purdue IT) and Graduate School helped fund PurdueThesis development.

TABLE OF CONTENTS

LIST OF TABLES 9

LIST OF FIGURES 10

LIST OF LISTINGS 11

LIST OF PROTOCOLS 12

LIST OF SCHEMES 13

LIST OF SYMBOLS 14

ABBREVIATIONS 15

GLOSSARY 16

ABSTRACT 17

1 INTRODUCTION 18

2 A SYSTEMATIC STUDY OF CRYPTOGRAPHIC FUNCTION IDENTIFICATION APPROACHES IN BINARIES 19

2.1 Introduction 19

2.2 Background 21

2.2.1 Categorization of detection approaches 22

2.2.2 Cryptographic Features 24

2.2.3 Applications with Modern Cryptographic Algorithms 26

2.3 Overview 26

2.3.1 Cryptographic Algorithm Detection Tools 26

2.3.2 Categorization of Cryptographic Algorithm Detection Tools 28

2.4 Analysis framework for open-source detection tools 29

2.4.1 Benchmarking Framework 29

2.4.2 Evaluation Metrics 32

2.5 Reproduction 33

2.5.1 Non-Reproducible Tools 34

2.6 Replication . . . 35
2.6.1 Non-Replicable Tools . . . 36
2.6.2 Unexpected Detection Results . . . 36
2.6.3 Operating System Limitations . . . 38
2.6.4 Impact of Compiler Variations . . . 38
2.6.5 Impact of Optimization Flags . . . 41
2.6.6 Impact of Different Algorithm Versions . . . 42
2.6.7 Impact of Intentional Evasion Tactics . . . 43
2.7 Discussion . . . 44
2.7.1 Approaches to Algorithm Detection in Tools . . . 44
2.7.2 AI/ML Approach . . . 45
2.7.3 Lack of Support for Modern Cryptographic Algorithms . . . 45
2.7.4 Comparative Detection of Similar Algorithms Across Various Tools . 46
2.7.5 User Experience . . . 47
2.7.6 Challenges for Future Research . . . 48
2.8 Conclusion . . . 50

3 SNARKPROBE: AN AUTOMATED SECURITY ANALYSIS FRAMEWORK FOR ZKSNARK IMPLEMENTATIONS . . . 51
3.1 Introduction . . . 51
3.1.1 Our contributions . . . 54
3.1.2 Chapter Organization . . . 54
3.2 Background and Related Work . . . 55
3.2.1 Background . . . 55
3.2.2 Related Works . . . 56
3.2.3 Challenges and motivation . . . 59
3.3 Overview . . . 59
3.3.1 Users and Use Cases . . . 59
3.3.2 Workflow . . . 60
3.4 Design and Implementation . . . 62
3.4.1 Constraint Checker . . . 63
3.4.2 SnarkFuzzer . . . 65
3.5 Performance Evaluation . . . 72
3.5.1 Performance of the Constraint Checker . . . 73
3.5.2 Performance of SnarkFuzzer . . . 74
3.6 Error Catching Evaluation . . . 78

3.6.1 Potentially Locatable in Current Libraries . . . 78
3.6.2 Found in Current Libraries . . . 80
3.6.3 Found in Previous Versions of Libraries . . . 84
3.7 Conclusion . . . 84

4 GREY-BOX DIFFERENTIAL FUZZING FOR ZERO-KNOWLEDGE PROOF BINARY APPLICATIONS . . . 85

5 ETHICS . . . 86

6 CONCLUSION & FUTURE WORK . . . 87

REFERENCES . . . 88

A Appendix for Chapter 2 . . . 96
A.1 List of Optimizations, Obfuscations, and their Combinations for Our Evaluation Metrics . . . 96
A.2 Procedure for R+R Evaluation . . . 96
A.2.1 Steps in our Reproduction Evaluation . . . 96
A.2.2 Steps in our Replication Evaluation . . . 97
A.3 Tools . . . 97
A.3.1 Aligot . . . 97
A.3.2 CryptoHunt . . . 98
A.3.3 CryptoKnight . . . 99
A.3.4 Draft Crypto Analyzer (DRACA) . . . 99
A.3.5 FindCrypt2 . . . 100
A.3.6 HCD . . . 100
A.3.7 PEiD KANAL . . . 100
A.3.8 Signsrch . . . 101
A.3.9 Where's Crypto . . . 101
A.4 Additional Tables . . . 101

B Appendix for Chapter 3 . . . 105
B.1 Additional Information for Constraint Checker . . . 106
B.1.1 An optimization for the SMT solver . . . 106
B.1.2 Protocol of the Constraint Checker by the SMT Solver . . . 108
B.1.3 Example Constraint Checker . . . 109

B.1.4 Domain Gap Issues 109
B.2 Additional Information for SnarkFuzzer 111
B.2.1 Example Ideal Model Files 111
B.2.2 PrettyPrint Example 112
B.3 Equations `comparison_gadget` Represents 113

LIST OF TABLES

2.1 Categorization of existing tools and their important properties 30

2.2 Reproducibility outcome for each tools. ✓ indicates a tool that has passed the reproduction evaluation, while ✗ represents a tool that has either failed the reproduction evaluation or is not accessible. − denotes a tool that is available for use, but the developer has not provided the original test cases. 34

2.3 Comparative analysis of detection approaches versus stated capabilities in original documentation. Cross mark denotes inconsistencies (false positive or false negative), check mark successful replication. 36

2.4 Tool compatibility with different operating systems; ●represents a tool that supports the desired system, while ○indicates a tool that does not support the desired system; ◐ represents cases where a tool cannot be executed on a Unix-like system but becomes functional with the assistance of WineHQ 39

2.5 Output when evaluating DRACA and Signsrch with different variants of TEA . 42

2.6 Evaluation results for cryptographic function detection in different malware versions 43

3.1 Comparison of `SNARKProbe` and a selection of cryptographic and generic fuzzing tools. A ◐ means achieves in select instances. 57

3.2 Constraint Checker runtime for different proof statements 73

3.3 Runtime (in seconds) of the Input Generator to produce R1CS matrices of different sizes. 76

3.4 Runtime of the Value Model and Value Monitor to process an R1CS matrix of varying size. For example, 10×30 is a matrix with 10 variables and 30 constraints. 78

3.5 Summary of the current potential errors and inconsistencies, and previous vulnerabilities, that `SNARKProbe` is able to catch and locate. 79

A.1 Performance Metrics for Tool Evaluation: False Positives, False Negatives, True Positives, and True Negatives 102

A.2 Cryptographic algorithm detection support as stated in the paper or documentation for each tool 103

A.3 Detailed evaluation results for each tool based on our uniform benchmarking suite 104

LIST OF FIGURES

2.1 Evolution of cryptographic function identification techniques 20

2.2 Timeline of the development of cryptographic algorithm detection tools 22

2.3 Types of cryptographic functionality and designs targeted by different tools and approaches, by percentage of tools surveyed 28

2.4 Evaluation result for MD5 with different compilers. A full marker indicates that function can be detected and an empty marker that it cannot. 39

2.5 Detection percentage by algorithm across all tools 46

3.1 Workflow of `SNARKProbe`. White boxes represent subcomponents universal for all zkSNARK libraries, forward diagonal stripes represent those designed for a specific library, and backward diagonal stripes represent those designed for a specific programming language. 60

3.2 Runtime of each component in SnarkFuzzer with different libraries and protocols, both in terms of actual time as well as percentage of the total. 75

3.3 Percentage (breadth) of library coverage after a given number of iterations . . . 76

B.1 Examples of Domain Comparison 111

B.2 Example file for the ideal branch file 111

B.3 Example file for the ideal value file 111

B.4 Content of variable type G1 112

LIST OF LISTINGS

LIST OF PROTOCOLS

LIST OF SCHEMES

LIST OF SYMBOLS

m mass

v velocity

ABBREVIATIONS

abbr	abbreviation
bcf	billion cubic feet
BMOC	big man on campus

GLOSSARY

Cryptographic Primitive	Cryptographic primitive (also called cryptographic function or cryptographic algorithm) refers to the entirety of a specific scheme or class of algorithms such as AES or RSA, used to generate ciphertext and support encryption and decryption processes for securing information.
Binary Analysis	Binary analysis is the examination of compiled executables without source code to understand program structure, behavior, and embedded functionality at the machine instruction level.
Cryptographic Logic Error	An error or bug that causes a program to produce an incorrect result due to errors in the cryptographic protocol implementation, such as incorrect mathematical operations or disregarding the specification.
Zero Knwoledge Proof	A cryptographic protocol that allows a user (the prover) proves a statement to another party (the verier) without revealing anything about the statement other than that it is true.
zk-SNARKs	Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge are a non-interactive zero-knowledge proof system with short proofs size and efficient verification, enabling practical deployment in cryptographic systems.
White-box Fuzzing	A testing approach where inputs are generated using full knowledge of the programs internal structure. Code and execution paths are analyzed to systematically explore behaviors and improve test coverage.
Black-box Fuzzing	A testing approach where inputs are generated without any knowledge of the programs internal structure. The system is treated as opaque, and only external behaviors such as crashes or errors are observed.

ABSTRACT

PurdueThesis is a LaTeX document class used for master's bypass reports, master's theses, PhD dissertations, and PhD preliminary reports. This template demonstrates how to use PurdueThesis.

1. INTRODUCTION

2. A SYSTEMATIC STUDY OF CRYPTOGRAPHIC FUNCTION IDENTIFICATION APPROACHES IN BINARIES

This chapter is based on joint work with Christina Garman (Purdue University) along with Priyam Biswas (Intel) [1]*. The paper was published in proceedings of the 2024 Annual Computer Security Applications Conference, pages 1092-1108 [1]* [2]*.*

2.1 Introduction

Cryptography has found itself at the core of secure modern technology and both plays a key role in securing communications and is imperative to many modern systems and applications. From data integrity to authentication and online banking to messaging friends, cryptography is everywhere. Well-known cryptographic libraries, such as OpenSSL [3], are widely used not only to generate TLS certificates and validate certificate information, but also to implement cryptography in general purpose applications. Despite their universal benign usage, cryptographic functions are also heavily used in malware and to evade security protocols. In recent ransomware attacks [4, 5], cryptographic functions have been used to encrypt a victim's information and later asked to pay ransom in exchange of recovering their files and information.

On the one hand, the usage of cryptographic functions makes it easier to carry out secure and private operations, and on the other hand, its malicious usage by bad actors makes it harder for cryptography and security experts to perform forensic analyses and reverse engineer code. Hence, the ability to automatically identify or detect the presence of cryptographic functions in binary applications can be a crucial part of the security experts' arsenal. It can assist in binary analysis to give a better depiction of how the functions work. Determining the type of cryptographic function in a binary can help to pinpoint the existence of a malicious payload [6] and also help cryptographic experts identify potentially insecure protocols or implementations (or those that require more manual analysis).

[1]↑Yongming Fan and Christina Garman's work was partially supported by NSF grant CNS-2047991.
[2]↑This paper also has an extended version presented at the 2024 Learning from Authoritative Security Experiment Results Workshop [2].

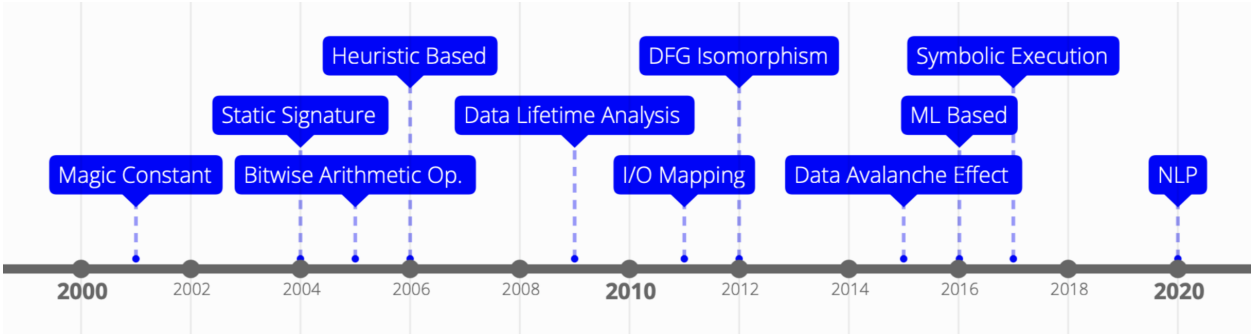


Figure 2.1. Evolution of cryptographic function identification techniques

While general purpose binary analysis and function identification techniques are themselves broad and thriving areas that could potentially solve these problems, a variety of work across industry and academia has focused on solving these challenges more specifically: developing techniques and tools that are uniquely tailored to identifying *cryptographic primitives* in binary applications. This allows researchers to often provide better results in different domains, leveraging cryptography-specific features. Cryptographic functions tend to have many mathematical computations, nested loops, and exclusive input-output mappings which are distinct from non-cryptographic functions and can thus be useful as a means of identification. Harvey et al. [7] first utilized magic constants to identify cryptographic primitives in 2001. Later works focused on signature based detection mechanisms. However, due to the limitations of signature based approaches, subsequent work placed a greater emphasis on heuristic based detection, such as identification of basic blocks, instruction patterns, etc. With the popularization of modern machine learning algorithms, newer works utilized deep learning and AI to extract cryptographic features from a binary.

Unfortunately, while there has been a wealth of work in the space, it has generally failed to keep up with modern cryptographic innovations and applications, as well as current software engineering and optimization techniques. This can be difficult to notice as there is no standardized benchmarking or comparison framework, and individual works tend to test on algorithms that they are best at identifying, while neglecting those they cannot. Additionally, as many techniques are tailored for specific cryptographic algorithms, architectures,

or settings, which change rapidly over time, reproducing past results can be challenging, making it difficult to provide comparisons across the growing landscape as well.

Given how important these tools and techniques can be for both researchers and practitioners, we aim to reproduce and replicate all existing tools to date in this domain. In this endeavor, we also offer a comprehensive analysis that categorizes and discusses the existing body of work, highlighting the strengths, weaknesses, and techniques employed. We also seek to help drive future research in the area by identifying potential challenges and various factors that may influence performance. To help provide for a more even comparison of existing and future work, and to identify gaps that are in need of future research, we create a benchmarking framework featuring a number of modern cryptographic primitives and applications. We then use this framework to evaluate existing work. Finally, throughout the paper, we discuss and motivate a variety of open research problems in the space.

Our contributions. Our primary contributions are:

- We conduct a thorough examination of cryptographic function identification methods, exploring the range of techniques utilized and evaluating their respective strengths and limitations.
- We create a standardized suite of performance metrics and benchmarks to evaluate the effectiveness of current detection mechanisms and analyze existing tools based on this suite.
- We conduct comprehensive replication and reproduction studies on all existing work.
- We provide an in-depth analysis of our findings and categorize the tools according to their performance across various dimensions.
- Based off of this analysis and our study of existing work, we discuss the research gaps in this domain and propose directions for future work.

2.2 Background

In this section, we provide background on and a classification of detection approaches that have been employed by existing tools to date. Figure 2.1 shows the evolution timeline of various techniques and approaches used in cryptographic function identification research

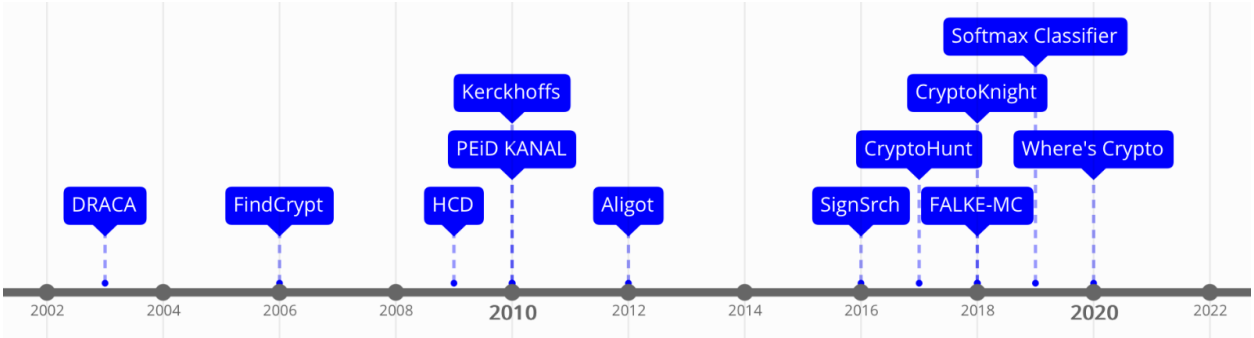


Figure 2.2. Timeline of the development of cryptographic algorithm detection tools

whereas Figure 2.2 shows the development timeline of all studied tools. We also provide an overview of some of the cryptographic features that existing tools focus on, to help better understand how they work and contextualize later discussion.

Cryptographic primitive vs. cryptographic function. A brief note on notation used throughout this paper. When we refer to a *cryptographic function* or *cryptographic algorithm*, we are generally referring to the entirety of a specific scheme or class of algorithms, such as RSA or hash functions. When we say *cryptographic primitive*, we generally mean a building block for a larger cryptographic scheme, such as a Feistel network.

2.2.1 Categorization of detection approaches

Precision, scalability and performance of an identification approach all rely on the underlying techniques used. Detection approaches can be divided into three categories: i) Dynamic, ii) Static, and iii) Machine learning based approaches.

Static Approaches. Static approaches perform various forms of static analysis to detect any static signatures, such as 'magic' constants, instruction sequences, and different code structures such as S-boxes, as a means to identify cryptographic algorithms. This type of approach is based on the assumption that cryptographic functions will perform a large number of arithmetic computations compared to non-cryptographic functions. Harvey et al. [7] first proposed the idea of identifying cryptographic algorithms in binary files. They focused on finding algorithms based on their constant characteristics. By taking advantage of fea-

ture matching, subsequent works [8–10] primarily focused on protocol reverse engineering. Chang et al. [11] created a library of 3000 plus signature characteristics of cryptographic algorithms and also explored recognizing cryptographic algorithms based on Minimum Perfect Hash Function (MPHF). Lestringant et al. [12] used data-flow isomorphism to find symmetric key algorithms.

Off-the-shelf tools such as Draft Crypto Analyzer (DRACA) [13], Kanal [14], Kerckhoffs [15], Hash & Crypto Detector (HCD) [16], Signsrch [17], and Findcrypt [18], utilize static signature patterns of different cryptographic functions. Static based approaches have low performance overhead. However, these detection mechanisms can be easily bypassed using simple obfuscation techniques. For example, the authors of Aligot [19] showed that simple data obfuscation techniques can bypass static analysis based detection mechanisms.

Dynamic Approaches. Dynamic approaches [19–21] primarily focus on identifying cryptographic primitives from execution traces. Lutz et al. [22] first applied dynamic analysis to identify cryptographic algorithms based on three indicators: i) presence of loops, ii) changes in entropy, and iii) high ratio of bitwise arithmetic instructions. Based on the data avalanche effect, CipherXRay [21] pinpoints the boundary of cryptographic operations and recovers transient cryptographic secrets. However, this approach does not work in the case of stream ciphers, for example, as they do not show any data avalanche effect.

Gröbert et al. [23] proposed heuristic based approaches on both generic characteristics of cryptographic code and on signatures for specific instances of cryptographic algorithms by mapping input-output (I/O) relations. Aligot [19] further extends this idea of I/O mapping. It retrieves I/O parameters in an implementation-independent fashion, and compares them with known cryptographic functions as well as performs an interloop data flow analysis. CryptoHunt [20] used bit-precise symbolic loop mapping to identify cryptographic functions and applies guided fuzzing to make the solution scalable. Park et al. [24] proposed a hardware-assisted tracing technique to detect symmetric-key cryptographic routines in anti-reverse engineered binaries via recording the change of flow instructions from the CPU at run-time. Dynamic approaches in general perform better than static based approaches in obfuscated binaries, but suffer from much greater performance overhead.

Machine Learning Based Approaches. With the growing popularity of machine learning, several recent works have explored utilizing such techniques. Shin et al. [25] discussed how recurrent neural networks can identify functions in binaries with greater accuracy and efficiency. However, their approach was generalized for any function identification. For the purpose of cryptographic function identification, Wright et al. [26] proposed an artificial neural network model to classify functional blocks as being either cryptographic or not by extracting the frequency of different logic instructions from a disassembled program. Benedetti et al. [27] used the 'grap' tool to detect cryptographic algorithms by creating patterns for AES and ChaCha20. Falke [28] proposed a neural network based approach by modeling classifiers for arbitrary cryptographic algorithms from sample files and then automatically extracting features to train the neural network. It offered a high detection rate in combination with a low false positive rate. Hill et al. [29] proposed a Dynamic Convolutional Neural Network based learning system (CryptoKnight) which learns from new cryptographic execution patterns to classify unknown software. Jia et al. [30] proposed an NLP-based approach which first extracts the semantic information of assembly instructions and then transfers them into 100-dimensional vectors and later uses K-Max-CNN-Attention to classify cryptographic functions.

2.2.2 Cryptographic Features

Cryptographic functions have their own (often quite distinct) set of characteristics, and work thus far has often leveraged these for identification, as they can lead to more tailored or performant techniques than general purpose binary analysis and code identification. Many of these features only pertain to certain algorithms and cannot be used to identify others. We now discuss some of the popular cryptographic features used for identification purposes.

Magic Constants. In this approach, algorithm specific "magic constants" are searched for in the binary. These are constant values that must occur in any implementation of a given algorithm. For instance, Signsrch [17] relies on the magic number '0x9e3779b9' to detect the TEA algorithm; however, it fails to identify the algorithm when data obfuscation is applied [20].

Presence of Loops. Most cryptographic functions use some form of loops for key generation, encryption, or other operations. CryptoHunt [20] utilized loops as a means for detection. Given that loops may present in regular functions, further program analysis is necessary to ensure accurate detection.

Changes in Entropy. The process of decryption usually decreases information entropy. Hence, entropy can be used as an indicator of encrypted versus plaintext data. Several techniques [22, 31] have used entropy to distinguish encrypted blocks from regular ones. However, relying on entropy can also lead to false positives in certain circumstances [23].

I/O Mapping. Cryptographic functions sometimes have one-to-one mappings between their input to output, i.e., given a key and the plaintext, we might always get the same output, or given the input to a hash function, we should always get the same output. However, these approaches rely on the accurate extraction of the key and input/output from the memory, as any wrong extraction can defeat the usefulness of unique mapping.

Data-Flow Isomorphism. In this approach [12], a Data-Flow Graph (DFG) [32] is generated from the corresponding assembly code, and subgraphs in the DFG are checked to see whether they are isomorphic to the graph signature of a particular cryptographic algorithm. However, this approach is limited in the case of conditional statements, which are more prominent in asymmetric algorithms. Additionally, DFG can vary in effectiveness depending on data obfuscations techniques used in the implementation, such as data-splitting.

S-Box. Substitution boxes (S-Box) [33] are a fundamental component used in many symmetric key ciphers. For instance, the BLOWFISH algorithm is a 64-bit block cipher that has an S-Box consisting of 1024 elements [30] which are usually sequentially stored in memory, allowing tools to detect BLOWFISH by locating this S-Box in memory. Similar techniques can be applied to Permutation Boxes (P-Box) [34], which are methods of bit-shuffling to permute or transpose bits across S-box inputs.

Instruction Sequences. An instruction sequence is a series of instructions or commands that a CPU executes in a specific order to perform tasks within a program. Many fundamental cryptographic operations will consist of fingerprintable instruction sequences and hence can be used for the purpose of identification. Nevertheless, this method may lose its effectiveness when dealing with control-flow obfuscations such as control-flow flattening [35].

2.2.3 Applications with Modern Cryptographic Algorithms

Transport Layer Security (TLS) is a cryptographic protocol that is specifically crafted to facilitate secure communication over networks.

Starting from TLS 1.1, elliptic curve cryptography (ECC) has been incorporated into the TLS protocol. Furthermore, in TLS 1.3, ECC takes a central role in the base specification, featuring new signature algorithms, while RSA support has been discontinued. As of 2021, the deprecation of TLS 1.0 and 1.1 has taken place, resulting in ECC as the major choice for TLS ciphersuites.

There are numerous libraries that implement TLS, including OpenSSL, cryptlib, and Java Secure Socket Extension. Concurrently, beyond the TLS, a number of other applications leverage modern cryptographic algorithms. Notable examples include iMessage and Signal, both of which implement end-to-end encryption for enhanced security.

2.3 Overview

This paper focuses on comprehensive replication and reproduction studies for existing work on identifying cryptography code in binary applications. To aid in this, we propose a novel analysis framework aimed at facilitating a comprehensive evaluation of cryptographic function detection tools' replicability. Our framework is introduced in section 2.4. Subsequently, we conduct two evaluations: a reproducibility evaluation in section 2.5, and a replicability evaluation in section 2.6. Additionally, we provide further discussion on the results of our reproducibility and replicability evaluations, as well as point out a number of important takeaways and directions for future work, in section 2.7. Finally, to support future research in this domain, we present a summary cryptographic detection approaches and tools in Section 2.2 as part of our background.

2.3.1 Cryptographic Algorithm Detection Tools

A number of cryptographic detection tools have been developed based on the different approaches discussed in Section 2.2.1. We perform our reproduction and replication studies on all existing work that we were able to find from 2000 until now:

- Aligot [19]
- CryptoHunt [20]
- CryptoKnight [29]
- DRACA [13]
- FindCrypt2 [18]
- FALKE-MC [28]
- HCD [16]
- Kerckhoff [15]
- PEiD KANAL [14]
- SignSrch [17]
- Softmax Classifier [30]
- Where's Crypto [36]

Our focus is on evaluating tools specifically designed for identifying cryptographic functions and comparing these tools with their stated performance. As such, we exclude other (broader) binary analysis tools, such as Binshape [37], a general-purpose binary analysis tool, or CIS [38], which focuses on testing cryptographic heuristics rather than actual algorithms.

We also employ a systematic approach to categorize, classify, and interrelate the diverse array of knowledge elements that constitute such tools from several directions in Section 2.3.2. Due to space constraints, we include brief discussions of each tool, along with their strengths and weaknesses, in Appendix A.3.

We comprehensively summarize the detection abilities of each cryptographic function detection tool. We did a thorough check of each tool's documents and signatures to see what types of cryptographic algorithms they can find. Figure 2.3 shows the distribution of various algorithms that existing work has focused on. After that, we gather all the information we found and explain it in more detail in Table A.2. This helps us better understand what each tool can do in terms of detecting different cryptographic algorithms.

Note that we compiled these results based on information provided by the developers of each tool. Through our comprehensive analysis, we found that commercial tools like DRACA [13], FindCrypt2 [18], and Signsrch [17] can detect a diverse range of cryptographic algorithms. However, their performance falter when encountering obfuscation or atypical situations. Certain academic-developed tools such as Aligot [19], CryptoHunt [20], and Where's Crypto [36] offer superior analysis results in specific scenarios, such as detecting cryptographic algorithms from obfuscated binaries. Many of them can only detect a few cryptographic algorithm, but they can be easily expanded.

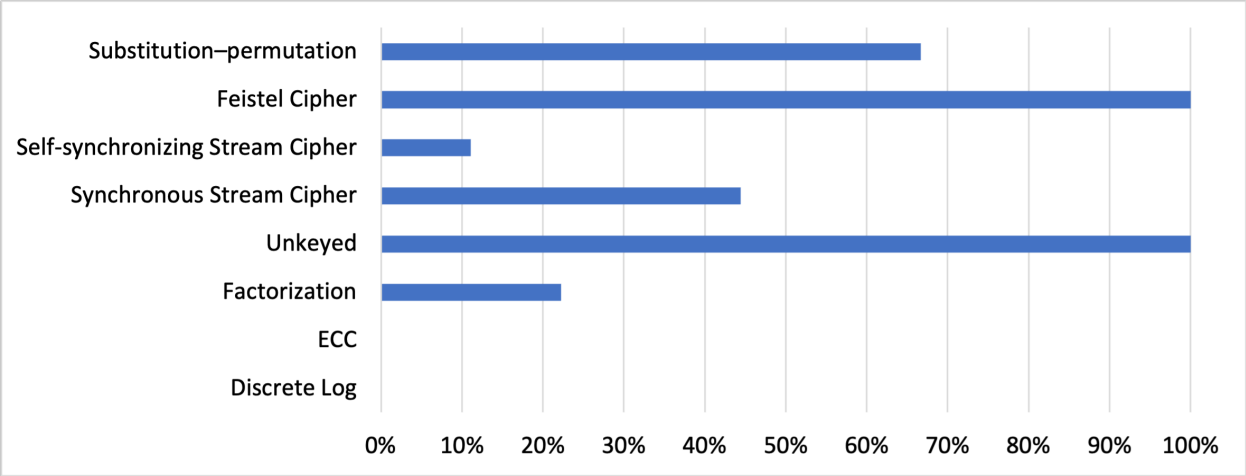


Figure 2.3. Types of cryptographic functionality and designs targeted by different tools and approaches, by percentage of tools surveyed

It is important to recognize that a simple summary may not fully capture the current capabilities and detection abilities of each tool. Moreover, many of these tools were developed years ago, and they may not be compatible with current programming languages, compilers, and/or hardware architectures. This is why our reproduction and replication study is so crucial. It not only highlights these major gaps in existing work as the space has progressed, but allows us to provide insights on the space more broadly, and on a variety of important avenues for future work.

2.3.2 Categorization of Cryptographic Algorithm Detection Tools

We employ a systematic approach to categorize, classify, and interrelate the diverse array of knowledge elements that constitute such cryptographic detection tools from several directions. Table 2.1 contains an overview and summary of each tool, including its reproduction and replication status in this work, as well as a variety of information such that one can easily find or organize existing (and future) tools based on the per-column categories.

We begin by categorizing the detection approach (broken down by the different methods discussed previously in Section 2.2.1) and the programming language in which the tool's

source code is written. We note that some tools are not open source; they provide only an executable file, and as a result, we lack information about the source language. Some tools require specific dependent libraries to run, which can limit or hinder their usability, and as such we list if there are any required dependencies. Some tools are installation-free and do not require compilation, offering an easy, user-friendly experience. And a few tools provide the capability for developers to insert new cryptographic algorithm signatures, thereby expanding their detection capabilities as the field advances.

2.4 Analysis framework for open-source detection tools

Previous tools have been evaluated only with certain basic cryptographic algorithms, and often seem to lack evaluation for false positives, real world applications, and large scale projects. Therefore, as the first step in our replication study, we build a new benchmarking framework with a number of evaluation metrics, which will not only encompass all evaluations conducted by previous tools but also establish a unified more comprehensive evaluation benchmark moving forward.

Yongming: add transaction

- Aligot [19]
- CryptoHunt [20]
- CryptoKnight [29]
- DRACA [13]
- FindCrypt2 [18]
- HCD [16]
- PEiD KANAL [14]
- SignSrch [17]
- Where's Crypto [36]

2.4.1 Benchmarking Framework

During our study, we identified a significant gap in the field: the absence of a standardized method for evaluating existing tools and their efficacy. We believe that to do a fair evaluation in any domain requires a standard benchmark. We propose a benchmarking framework that will help us understand the scalability and effectiveness of any given detection approach. We have four categories in our framework: i) basic cryptographic functions, ii) microbenchmarks, iii) libraries, and iv) large projects. Our framework is flexible and can be easily updated as the field of cryptography advances.

Tool	Method	Dependent Library	Language	Requires Compiling	Expandable	R+R	Obf. Binary Design	AI/ML Approach
Aligot	I/O Parameters	Intel Pin	Python	Yes	Yes	Reproducible	No	No
CryptoHunt	Bit-precise Symbolic Loop	Intel Pin	C++	Yes	Yes	Unable	Yes	No
CryptoKnight	Machine Learning Based	Intel Pin, PyTorch	Python	Yes	Yes	Both	No	Yes
DRACA	Unknown	None	Executable	No	No	Replicable	No	No
FindCrypt2	Magic Constants	IDA Pro	C++	Yes	No	Replicable	No	No
FALKE-MC	Neural Network	None	Unknown	Unknown	Unknown	Unable	Yes	Yes
HCD	Unknown	None	Executable	No	No	Replicable	No	No
Kerckhoff	Multiple	Intel Pin	Python	No	Yes	Unable	No	No
PEiD KANAL	Signature Searching	None	Executable	No	Yes	Replicable	No	No
Signsrch	Signature Searching	None	Executable	No	No	Replicable	No	No
Softmax Classifier	Neural Network	Unknown	Python	Unknown	Unknown	Unable	No	Yes
Wheres Crypto	DFG Isomorphism	IDA SDK	C++	Yes	Yes	Both	No	No

Table 2.1. Categorization of existing tools and their important properties

Basic cryptographic functions. The basic cryptographic functions category contains cryptographic algorithms from various standard classes within the cryptographic space. A single cryptographic algorithm may have multiple versions, or may use non-standard implementations, so we may select many different implementations or versions for certain algorithms. The cryptographic algorithms we select are:

- AES256
- DES
- ECC
- MD5
- RC4
- RC5
- RSA
- SHA1
- SHA256
- TEA
- XTEA
- XXTEA

We have chosen these algorithms for several reasons. First, we have included those recommended by the National Institute of Standards and Technology (NIST) [39], as they are widely utilized and cover a broad range of real-world applications. Second, we have incorporated algorithms like TEA and MD5, which, although once popular, are now considered unsafe or outdated, but still might have historical significance in applications like malware. Last, we have selected various variants of a single cryptographic algorithm to assess a tool's ability to handle different implementations effectively.

Microbenchmarks. Our microbenchmarks category contains small programs on file manipulation, networking, I/O heavy programs, math heavy programs, matrix and array, and so-called "golden implementations" of well-known cryptographic algorithms. We have chosen small programs such as I/O heavy and math-heavy programs since they have similar instruction sets or behavior patterns as cryptographic functions. This allows us to test for false positives in a detection framework.

Libraries. We have chosen cryptographic libraries as well as encoding and compression libraries that have similar behavior to cryptographic functions including:

- openssl (3.3.1)
- libgcrypt (1.8.11)
- libsodium (1.0.20)
- mbedTLS (3.6.1)
- gnuTLS (3.8.6)
- bzip2 (1.0.8)
- zlib (1.3.1)
- ffmpeg (7.0.2)
- libgsm (1.0.17)
- libjpeg (6b)
- libpng (1.6.43)

This allows us to again test for false positives, but also to test for a variety of different cryptographic algorithms implemented in different ways. We select open source libraries only, so that we are able to compile them based on our evaluation metrics. For each tool, we used the latest version available at the time of evaluation. Our selection includes a diverse range of libraries, from those with frequent recent updates, such as OpenSSL, to those that are a bit older, such as libjpeg.

Large projects. We also select Signal, an application designed for encrypted messaging, to evaluate a tool's ability to both scale for large codebases as well as still detect cryptography within them. We believe that to understand the scalability of a mechanism, it is crucial to determine that the tool performs well irrespective of the code size and its applications. Signal was selected because it both utilizes multiple (modern) cryptographic schemes and is open source. This again means we can recompile Signal with our evaluation metrics, plus establish ground truth for testing. However, as our framework is open-source, one could easily extend it to include any future desired large-scale open-source projects, such as Tor.

2.4.2 Evaluation Metrics

To ensure a thorough exploration of current tools (and to provide rigorous and fair evaluation metrics for the future), we have meticulously crafted a series of evaluation experiments. These experiments are thoughtfully designed to align with the challenges we discuss in Section 2.7.6, maximizing our ability to uncover the current state of these tools and elucidate their strengths and weaknesses. Our evaluation metrics are as follows:

- **Based on optimization levels:** Evaluate based on different compiler optimizations including -O0, -O1, -O2, -O3, -Os, and -Ofast
- **Based on different combinations of obfuscation mechanisms:**
  - **Control-flow obfuscation:** Adding bogus control-flow, control-flow flattening, substitution of instructions, polymorphism
  - **Data-flow obfuscation:** Data aggregation, data-splitting, variable transformation

- **Modified versions of algorithms:** For example, modified TEA (Russian TEA) used in malware as suggested in [20]
- **Layout Obfuscation:** Address obfuscation, obfuscating debug information, address layout/memory layout randomization
- **Combination of all prior techniques**

- **Based on different compilers:** Evaluate based on different compilers, including GCC, CLANG, MSVC

The full list of optimizations, obfuscations, and their combinations we select for our evaluation can be found in Appendix A.1.

We assessed each tool with these benchmarks and metrics to thoroughly explore accuracy and detection capabilities when handling binary programs subjected to specific compiler or optimization/obfuscation techniques.[3] For instance, a tool **A** might successfully detect MD5 when all optimization flags are applied but fail to do so in the presence of obfuscation. Conversely, a tool **B** may be able to detect TEA when compiled with GCC but not when compiled with CLANG. This exploration provides us with a more comprehensive understanding of the current state of cryptographic function detection tools, revealing their weaknesses and indicating areas for future development from various perspectives.

2.5 Reproduction

We start by assessing the reproducibility of each tool by employing consistent experimental setups, procedures, and operating conditions to replicate their purported capabilities.

In our reproducibility evaluation we follow the ACM guidelines on reproducibility [40], using the same measurement procedure, system setting, and operating conditions as the tool's original test cases[4]. This will help us to understand the basic fundamental reliability for each tool. When tools do not pass the reproduction evaluation, we also provide a brief discussion for why. Table 2.2 shows the reproducibility results.

[3]↑All benchmarks and metrics we evaluate are available at https://github.com/BARC-Purdue/CryptoBinary.
[4]↑For the precise details and steps of our reproducibility evaluation see Appendix A.2.1.

Aligot	Crypto Hunt	Crypto Knight	DRACA	FindCrypt2	FALKE-MC
✓	✗	✓	–	–	✗
HCD	Kerckhoff	PEiD KANAL	SignSrch	Softmax Classifier	Wheres Crypto
–	✗	–	–	✗	✓

Table 2.2. Reproducibility outcome for each tools. ✓ indicates a tool that has passed the reproduction evaluation, while ✗ represents a tool that has either failed the reproduction evaluation or is not accessible. – denotes a tool that is available for use, but the developer has not provided the original test cases.

Note that for certain tools, the developers only provided the executable binary program without the accompanying source code or test cases necessary for evaluation, making a reproducibility assessment unfeasible (denoted –). Some tools are not publicly available in either source code or executable binary form, making a reproducibility evaluation impossible. As a result, they are marked with a ✗. Additionally, we encountered crashes when running Pin-based tools with the latest version of Intel Pin, indicating that the design of older cryptographic detection tools may not be compatible with newer Pin versions. However, Aligot is the one exception to this. Aligot provides step-by-step test cases, and, based on this, we have successfully reproduced each step with the expected results. Therefore, we consider Aligot reproducible, as we were able to replicate the test results using their original measurement procedures.

2.5.1 Non-Reproducible Tools

CryptoHunt has been successfully executed, but we are unable to provide the complete benchmarking framework and performance evaluation results. CryptoHunt detects cryptographic functions in binary code through a bit-precise symbolic loop mapping approach. However, depending on the binary's size, CryptoHunt may identify up to 10,000 loops within the binary. Without additional filtering, the process of comparing each of these loops with reference loops becomes exceedingly time-consuming and infeasible.

Kerckhoff cannot be evaluated in this paper due to its incompatibility with existing available versions of the Pin tracing tool, which cause it to crash. We have encountered a similar issue in the replication evaluation as well, which will be discussed in section 2.6.

Softmax Classifier and **FALKE-MC** cannot be evaluated in this paper since they are not available for public use, as neither the source code nor the compiled binary is provided. Only the original research paper is available, with no accompanying software or executable.

2.6 Replication

We now undertake a comprehensive replicability evaluation of existing work across different categories of cryptographic algorithms with our newly proposed analysis framework, in order to understand their consistency, robustness, and effectiveness. Our replication evaluation follows the ACM guidelines on replicability [40]. We rebuild each tool (if the source code is available) and run it across different system environments using our newly developed evaluation benchmark as the test cases[5]. This evaluation encompasses all work across this domain, allowing us to delve into their performance and potential limitations. For each cryptographic algorithm, we considered different optimization/obfuscation levels, compilers, and implementations. These experiments are designed to help us understand the consistency and robustness of existing work and techniques, the current status of cryptographic function detection performance and to explore potential future research questions, as well as to highlight existing challenges in cryptographic function detection. We also involve micro-benchmarks, libraries, and large scale projects to examine each tool's performance in real-world applications. We present the full results of our evaluation in Table A.3 in Appendix A.4, and discuss a selection of the results, as well as their implications, now.

Based on our replicability evaluation, we find that some tools cannot be replicated with our framework. We also highlight some interesting new results, which may help direct future research in this field. This will be addressed in both this section and in Section 2.7.

[5]↑For the precise details and steps of our replicability evaluation see Appendix A.2.2.

Tool	Cryptographic Algorithm								
	AES	DES	MD5	RC4	RC5	RSA	SHA1	SHA256	TEA
DRACA	✗	✗	✓		✓		✓		✓
CryptoKnight	✓		✗	✗		✓			
FindCrypt2	✗	✓	✓		✓		✓	✓	✓
HCD	–	–	–	–	–	–	–	–	–
PEiD KANAL	–	–	–	–	–	–	–	–	–
SignSrch	✓	✗	✓		✓		✓	✓	✓
Where's Crypto	✓	✓	✓				✓	✓	✓

Table 2.3. Comparative analysis of detection approaches versus stated capabilities in original documentation. Cross mark denotes inconsistencies (false positive or false negative), check mark successful replication.

2.6.1 Non-Replicable Tools

Aligot is a cryptographic function identification tool developed based on Pin, a dynamic binary instrumentation framework developed by Intel. We successfully reproduce the test cases that Aligot provided. However, Aligot was tested with Pin 2.10 and Pin 2.12 by its developer, but unfortunately, these versions are no longer available. Consequently, we attempted to use an older version of Pin (3.22) but we were unable to execute Aligot correctly. We believe this to be due to the fact that this tool does not support and cannot be executed on recent CPU architectures.

2.6.2 Unexpected Detection Results

We begin by discussing evaluation results that were either unexpected or inconsistent based on prior stated findings. During our evaluation, we encountered instances where certain tools yielded false positives or false negatives in their detection outcomes. These findings underline the importance of a more in-depth and rigorous analysis to ensure the attainment of dependable detection results. As a result, it is evident that further development is required to enhance reliability.

Unreplicable cryptographic function detection. In Table A.2, we list the purported performance for each tool. However, our evaluation could not verify the detection capabilities for certain cryptographic functions in some of the tools. Table 2.3 presents the outcomes

of our assessment relative to the detection capabilities as claimed by the respective tools' original documentation. If a tool claims to have the ability to detect a cryptographic algorithm, but we cannot reproduce the same result in any of our benchmarks or evaluation metrics, we mark it with a red exclamation mark. Otherwise, if we can confirm the detection ability, we mark it with a green circle. A blank means the tool makes no claim about the given algorithm. Due to the unavailability of documentation for HCD and PEiD KANAL, a comparison of our evaluation results with their claimed detection capabilities was not possible.

Micro-benchmark false positives. In our micro-benchmark evaluations, we also uncovered new false positive results. None of the tools falsely considered operations like file handling, I/O, network connection, or heavy math operations as cryptographic functions. However, some tools, such as CryptoKnight and Signsrch, mistakenly identify a mathematical matrix as a cryptographic function like AES. Since AES includes the Rijndael S-box (often coded as a matrix), tools that rely on static signature searches may incorrectly recognize a matrix as an AES function. This highlights the importance of broader and more standardized test cases as well, as things like this were not considered by previous evaluations. To better understand these results, in Table A.1 (Appendix A.4) we present the detailed breakdown of percentage of false positives and false negatives observed for each tool across various cryptographic functions, offering a comprehensive comparison of their performance in these error metrics. This allows us to gain insight into the likelihood of a tool raising errors when attempting to detect specific cryptographic functions.

Undetected cryptographic functions in libraries. Finally, besides standard standalone cryptographic functions, we also selected some real-world cryptographic libraries and libraries with similar behavior to cryptographic functions to test on. We find that many tools successfully detect cryptographic functions in these libraries, which is expected, as some, including openssl and libgcrypt, are indeed primarily cryptographic libraries. However, it is interesting to note that this is not true of all tools. Some work fails to detect cryptographic functions in this setting. For example, tools like DRACA, HCD, and Signsrch are unable to detect any cryptographic functions from openssl, while CryptoKnight and HCD fail to detect cryptographic functions in libgcrypt. We are not sure why this is the case, and leave

this as an interesting avenue for future work. This also points to the fact that existing work may have unreliable performance in real world (large) programs.

Future Research: Certain cryptographic-specific libraries seem to pose challenges for existing tools; exploring why this is the case would be an interesting avenue for future work. Also, existing tools have more false positives/negatives than initially expected, so placing more emphasis on improving detection accuracy and reducing false positives or false negatives seems highly relevant.

2.6.3 Operating System Limitations

The usage of certain tools may be constrained by their exclusive compatibility with specific operating systems, impacting their efficacy and analysis capabilities. Many tools are designed and built with specific operating systems in mind. Fortunately, with the help of other tools, such as WineHQ, which enables software developed for Microsoft Windows to run on Unix-like operating systems, developers can for example use some Windows cryptographic detection tools in Linux. However, tools are not just limited to running on specific operating systems, but also restricted to handling binaries compiled for selected operating systems. These inherent limitations potentially reduce the scope and depth of analysis a tool can provide, depending on the operating system selected and targeted in the original work.

In Table 2.4, we evaluate the capability of exisiting work to analyze binaries across three different operating systems and for binaries compiled specifically for each of these operating systems. We use a ◐ to indicate cases where a tool cannot be executed on a Unix-like system but becomes functional with the assistance of WineHQ.

Takeaway: Ideally cryptographic function detection tools need to support all operating systems to effectively detect cryptographic functions or malware across diverse environments and improve overall detection measures.

2.6.4 Impact of Compiler Variations

To study the impact of compilers on cryptographic function detection, we compiled each program in our benchmarking framework with GCC/G++, CLANG/CLANG++, and

Tool	Supported System			Supported Binary		
	Windows	macOS	Linux	Windows	macOS	Linux
Aligot	●	●	●	●	●	●
CryptoHunt	○	○	●	○	○	●
CryptoKnight	●	●	●	●	●	●
DRACA	●	◐	◐	●	●	●
FindCrypt2	●	●	●	●	●	●
HCD	●	◐	◐	●	○	○
Kerckhoff	●	○	○	●	○	○
PEiD KANAL	●	◐	◐	●	●	●
Signsrch	●	◐	◐	●	●	●
Wheres Crypto	●	○	○	●	●	●

Table 2.4. Tool compatibility with different operating systems; ●represents a tool that supports the desired system, while ○indicates a tool that does not support the desired system; ◐ represents cases where a tool cannot be executed on a Unix-like system but becomes functional with the assistance of WineHQ

MSVC. Somewhat as expected given the impact a compiler can have on the resulting binary, our evaluation shows that tools may output quite different results for different compilers. We choose MD5 as an illustrative example, shown in Figure 2.4, though we have observed the same general trends across most cryptographic schemes.

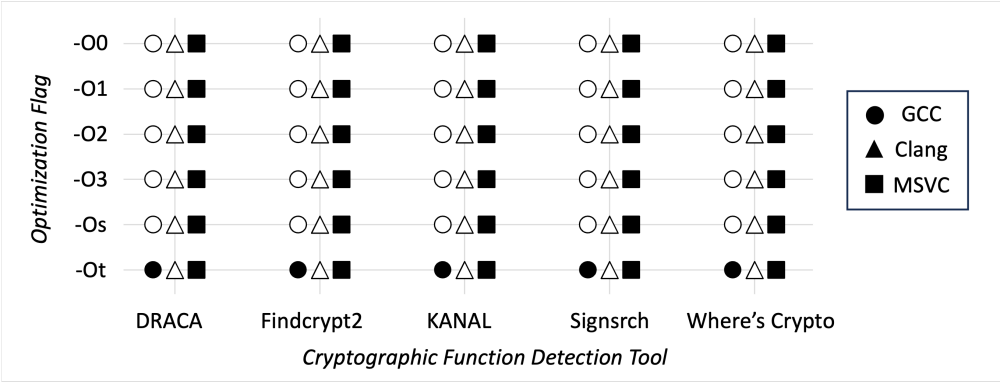


Figure 2.4. Evaluation result for MD5 with different compilers. A full marker indicates that function can be detected and an empty marker that it cannot.

Yongming: fix typo in figure!!!

In this example, DRACA, Findcrypt2, PEiD KANAL, Signsrch, and Where's Crypto support detecting the MD5 algorithm. However, their detection capabilities are influenced by the compiler used. All of these tools can detect the MD5 algorithm in binaries compiled with the MSVC compiler, regardless of the optimization flag. In contrast, for binaries compiled with GCC, they can only detect the MD5 algorithm when the -O0 optimization flag is applied. Furthermore, when using the Clang compiler, none of these tools were able to detect the MD5 algorithm in the binary. This suggests that differences between compilers may significantly impact a tool's analysis results. In general, we found that the MSVC compiler has less impact on detection results compared to GCC and CLANG. Specifically, if a tool can detect a certain cryptographic algorithm in a program compiled by GCC or CLANG, it will also be able to detect that algorithm in the same program compiled by MSVC.

For example, Signsrch shows inconsistent performance with Clang, GCC, and MSVC, identifying the SHA-1 hash function in certain programs compiled with identical optimization flags but using a different compiler. We also observe same behaviour for DARACA when detect XXTEA algorithm (see Table A.3 for details).

The structure of binaries produced by different compilers, such as MSVC, GCC, and LLVM, can vary significantly due to differences in object file formats, optimization strategies, and code generation techniques. Binary analysis often relies on symbolic information, padding, and inlined data, all of which are heavily influenced by the compiler version [41]. These variations can hinder the effectiveness of cryptographic function detection tools, potentially leading to inaccurate or incomplete detection results.

Future Research: At present, it is notable that none of the cryptographic function detection tools consider the impact of the compiler. Our evaluation results have revealed that the choice of compiler plays a pivotal role in cryptographic function analysis. The exploration of compiler effects and their incorporation into cryptographic function detection tools represents an enticing avenue for future research. Moreover, understanding and accommodating these compiler-related influences could significantly enhance both accuracy and reliability.

2.6.5 Impact of Optimization Flags

Optimization flags and obfuscation can significantly impact the analysis results for cryptographic functions [42]. CryptoHunt[20] and FALKE-MC [28] have explored this direction. CryptoHunt primarily focuses on specific optimization flags such as `-O0`, `-O1`, and `-O2`, while FALKE-MC includes programs compiled with `-Os` and `-Ofast` for training. However, we are unable to run either of these tools, so we cannot confirm their detection performance with optimization/obfuscations flags. In order to comprehensively investigate this topic, we have expanded our study to include additional optimization flags, obfuscation techniques, and their combinations, thereby providing a more thorough exploration of their impact on cryptographic algorithm detection.

Our findings indicate that both optimization flags and obfuscations can affect detection capability. For instance, based on our results in Table A.3, we can see that Findcrypt2, PEiD KANAL, Signsrch, and Where's Crypto can detect MD5 when compiled without optimization/obfuscation flags or compiled with `-O1`. However, none of these tools can detect MD5 if the binary program is compiled with advanced optimization/obfuscation flags such as `-O2`, `-Ofast`, or `-obs_sub`. In addition, we observed that Signsrch exhibits inconsistent performance for SHA-1 or SHA-256, and DRACA shows unstable performance with XTEA and XXTEA when running with optimization or obfuscation flags. Optimization flags significantly alter the structure and behavior of a binary program by improving performance, reducing size, and modifying control flow. Techniques like function inlining, loop unrolling, and dead code elimination obscure the original source code structure, making it more difficult for analysts to identify cryptographic functions.

Future Research: The effects of optimization and obfuscation have been noted in works such as CryptoHunt, which is designed specifically for obfuscated binaries. However, limitations such as slow performance due to the large trace size, may reduce its applicability for large scale projects. Considering that many tools seem to face challenges with obfuscated binaries, we assert that investigating the effects of optimization and obfuscation continues to be a compelling and critical avenue for future research.

2.6.6 Impact of Different Algorithm Versions

We also carried out evaluations to determine whether variations in a cryptographic algorithm affects the success of its detection. As a case study, we analyzed different variants of the Tiny Encryption Algorithm (TEA), including standard TEA, XTEA, and XXTEA. While all the tools were able to successfully detect TEA binaries with any of these three variants, we observed some interesting details in the detection results.

Algorithm	DRACA	Signsrch
TEA	* RC5 or RC6 - 50% ** TEA - 67%	* TEA1_DS [32.le.4] ** TEA encryption/decryption
XTEA	* RC5 or RC6 - 50% ** TEA - 34%	* TEA1_DS [32.le.4]
XXTEA	* RC5 or RC6 - 50% ** TEA - 67%	* TEA1_DS [32.le.4]

Table 2.5. Output when evaluating DRACA and Signsrch with different variants of TEA

For example, as shown in Table 2.5, when using DRACA to analyze binaries containing TEA or XXTEA, the tool provides a confidence score. It tends to determine that a binary program is more likely to contain TEA functions if TEA or XXTEA is present. However, when analyzing binary programs containing XTEA, DRACA may incorrectly identify RC5 or RC6 as being present in 50% of cases, and TEA functions in only 34% of cases. In contrast, Signsrch is able to detect TEA functions in all three binary programs containing TEA, XTEA, or XXTEA, but it may identify different TEA signatures in each binary.

Future Research: Our evaluations indicates that the variant or version of a cryptographic algorithm plays a crucial role in its detection, suggesting that existing tools might struggle to accurately identify certain variants based on their signatures alone. Currently, there is no approach that focuses on this difference, which could be an interesting future research path.

2.6.7 Impact of Intentional Evasion Tactics

We selected multiple (ransomware) malware binaries, including CryptoLocker [43], CryptoWall [44], Locky [45], TeslaCrypt [46], and Win32Dircrypt [47], to evaluate if cryptographic detection tools are better or worse at finding cryptographic algorithms in malware binaries (where authors might intentionally try to obfuscate the presence of cryptography) versus normal applications.

We tested the tools with these malware samples and present the results in Table 2.6. It is worth noting that malware may have different versions depending on the discovery date, and detection results are highly dependent on these versions. We found that the tools could detect cryptographic functions in TeslaCrypt version 2 but failed to do so in versions 1 and 3. This suggests that the malware may employ non-standard algorithms to evade binary analysis in certain versions, increasing the difficulty of detection. Furthermore, cryptographic functions in CryptoLocker's September 10, 2013 version were detectable by the tools, but in the November 20, 2013, and January 22, 2014 versions, they were no longer detectable. In the latter two versions, some tools identified the presence of anti-debugging techniques in the malware, which might explain why cryptographic functions could not be detected anymore.

Malware	DRACA	PEiD KANAL	Signsrch	Note
CryptoLocker_10Sep2013	✗	✓	✓	
CryptoLocker_20Nov2013	✗	✗	✗	anti-debug
CryptoLocker_22Jan2014	✗	✗	✗	anti-debug
CryptoWall	✗	✗	✗	anti-debug
Locky	✗	✗	✗	
TeslaCrypt_v1	✗	✗	✗	
TeslaCrypt_v2	✓	✓	✓	
TeslaCrypt_v3	✗	✗	✗	
Win32Dircrypt	✗	✓	✓	

Table 2.6. Evaluation results for cryptographic function detection in different malware versions

Future Research: Cryptographic function detection can be a useful component in identifying malware, and according to our investigation, current tools do not consistently detect cryptographic functions within the malware samples. This challenge has been noted in other research as well, for example, CryptoHunt identified environment-sensitive malware as a potential source of false negatives in detection efforts. Therefore, cryptographic function detection primarily focused on targeting malware could be a highly relevant source of future research in this area.

2.7 Discussion

We now provide a discussion backed by our evaluation results and examination of existing work and implemented tools, delving into broader issues that we discovered. This includes discussing additional overarching future research directions and key areas where improvements can be made moving forward.

2.7.1 Approaches to Algorithm Detection in Tools

We start by discussing the detection results of a single tool for different algorithms. We opted to evaluate 12 distinct algorithms (AES, DES, ECC, MD5, RC4, RC5, RSA, SHA-1, SHA-256, TEA, XTEA, and XXTEA) along with our microbenchmarks, various libraries, and a large-scale project.

Based on our evaluation (see Table A.3), we found that PEiD KANAL, SignSrch, and Where's Crypto can detect most algorithms we selected. This was somewhat expected as Where's Crypto is a more modern detection tool. HCD demonstrated average performance in our evaluation results. Nonetheless, we came across cases where tools did not detect cryptographic functions they were expected to identify, indicating potential need to enhance detection accuracy.

Future Research: Current tools seemingly employ various (often distinct) approaches compared to one another. Many of these tools also provide developers with the opportunity to expand and enhance their detection capabilities. Therefore, future research could focus on improving existing technologies within individual tools and existing approaches to achieve broader applicability with higher detection rates, and to make it so that users do not need to use different tools to best identify different algorithms.

2.7.2 AI/ML Approach

Artificial intelligence and machine learning (AI/ML) have been extensively applied across various domains to address complex problems. Previous works, like CryptoKnight, have utilized machine learning techniques for cryptographic function detection. However, in our evaluation, we found that these tools did not exhibit satisfactory performance. We attribute this issue not to the approach itself, but rather to the fact that these tools typically only offer a demonstration to showcase the potential application of AI/ML in this field. The main problem lies in the lack of sufficient signatures or analysis provided by these tools, which is likely the primary reason for their low performance.

Future Research: While current AI/ML based tools demonstrate the feasibility of utilizing AI/ML in cryptographic function detection, they fall short in providing the robust and comprehensive signature or analysis capabilities necessary for accurate detection. These seem like promising future research directions.

2.7.3 Lack of Support for Modern Cryptographic Algorithms

Many modern cryptographic algorithms, such as general Elliptic-Curve Cryptography (ECC) and more specifically ECDSA and ECDHE, have found widespread use in diverse applications [48] due to their advantages in terms of smaller private key size and improved efficiency at equivalent or higher security levels. ECDSA is utilized in a number of areas, including electronic healthcare, banking, commerce, and vehicles [49], and is currently supported by most major libraries, including cryptlib, Crypto++, libgcrypt, LibreSSL, and OpenSSL. However, our evaluation reveals that existing work lacks the ability to find such

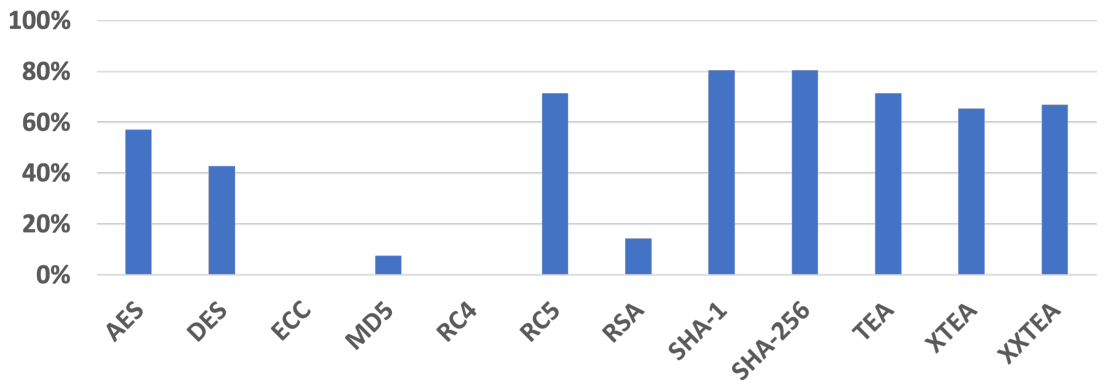


Figure 2.5. Detection percentage by algorithm across all tools

modern algorithms. As such, this could be a rich area for future development and improvement.

Future Research: As the prevalence of applications adopting ECC continues to rise, the ongoing development of cryptographic function detection has led to a disconnect between the outcomes of these efforts and the real-world applications that people are eager to test, leading to an important direction for future research.

2.7.4 Comparative Detection of Similar Algorithms Across Various Tools

Our evaluation helps provides insights from an algorithmic perspective on how well different tools can detect the same cryptographic algorithm. In our evaluation, we test the tools' abilities to find cryptographic functions in different settings (optimization flags, compilers, etc.). Figure 2.5 shows the percentage of algorithms that can be successfully detected by a tool across all evaluation experiments that we ran. We can see that TEA, RC5, and SHA can be successfully detected in most experiments. However, AES, and MD5 can only be found in a few experiments, with algorithms like RC4 and ECC not even being detected at all.

From our observations, it is not evident that there exists a consistent rule governing the ease of detecting specific categories of cryptographic functions. For instance, when considering two hash functions, MD5 and SHA, we note markedly different results. Even within the same family we find variations; TEA and RC5, both belonging to the block cipher

family, exhibit detectability, while DES, another block cipher, proves more challenging to detect.

Future Research: Some cryptographic algorithms appear to be more challenging to detect than others. However, we have not observed a distinct pattern suggesting that algorithms within a specific category consistently present greater difficulty in detection. Therefore, future research could focus on how to detect cryptographic algorithms that appear to be harder to identify or investigate the underlying factors that contribute to variations in the detectability of algorithms within the same class.

2.7.5 User Experience

Considering the experience and insights we have garnered from our evaluation process, we believe that discussing the developer experience is also an intriguing topic, in terms of how developers interact with these tools and their ease of use.

Older cryptographic detection tools, such as DRACA, PEiD KANAL, and SignSrch, do not require developers to compile and build the tools from source code. Developers can readily utilize these tools with supported operating systems. However, when compared to some better performing tools, these user-friendly alternatives exhibit lower performance and utility. For example, obfuscation can disturb the binary analysis process of Signsrch, leading to false negatives in the detection of certain cryptographic algorithms. Furthermore, DRACA struggles to accurately analyze packed executable programs, and consequently, it can only provide a rudimentary analysis outcome, despite its ease of use.

In addition to the aforementioned challenges, some tools pose difficulties for users due to technological complexity or data-related issues. Take, for instance, IDA Scope, Findcrypt2, and Where's Crypto, which all rely on IDA Pro. IDA Pro is not a free, open-source software, and these tools cannot be utilized by developers without it. Conversely, tools such as CryptoKnight and other machine learning-based techniques require users to provide substantial amounts of data, which further compounds the difficulty of using these tools.

However, these tools generally exhibit superior performance when compared to user-friendly alternatives. For instance, CryptoHunt consistently yields accurate analysis results

for identification within obfuscated programs. Where's Crypto, one of the most recent cryptographic detection tools, broadens the scope of analysis to encompass unfamiliar and proprietary cryptographic primitives, without relying on heuristics to select code fragments.

2.7.6 Challenges for Future Research

Based on our evaluation, we have identified several challenges that make cryptographic function identification difficult. In this subsection, we discuss some of these challenges, as we believe they may help inform future tool development and areas of research.

Obfuscation. One of the major challenges in cryptographic function identification is obfuscation. The initial purpose of obfuscation was to protect software intellectual property [50] from malicious reverse engineering attempts. However, malware authors adopted obfuscation techniques as a way to avoid detection. Various types of obfuscation techniques have been implemented to thwart any form of detection. These obfuscation techniques can be primarily categorized as: control-flow obfuscation, data obfuscation, and layout obfuscation. For a variety of reasons, one might still wish to identify cryptographic code even in the face of such obfuscation techniques, particularly given its prevalence nowadays.

Control-flow obfuscation is the most common type of obfuscation. A control-flow graph (CFG) [51] embodies the graphical representation of the flow of a program. To hinder CFG-based detection, malware authors introduce techniques such as including bogus control-flow [52], control-flow flattening [35], and opaque predicates [53]. Similar to control flow obfuscation, data obfuscation techniques, like data aggregation and data splitting, attempt to hinder approaches based on input/output relationships. For example, one might split a single variable into two to prevent detection. Layout obfuscation [54] scrambles the layout of a program's instructions while keeping the original syntax intact. Layouts can be obfuscated by adding meaningless classifiers, stripping redundant symbols, separating related code, as well as adding junk code.

Implementation Variation. Despite having a well-defined specification, cryptographic algorithms can be implemented in a number of ways while still achieving the same result. In addition, cryptography is not always implemented correctly or exactly to spec. For ex-

ample, buggy implementations of the TEA algorithm were found [19] in malware such as Storm Worm and Silent Banker. Hence, even if a detection approach can find an ideal implementation of an algorithm, there is no guarantee it can also detect all the implementation variations of the same algorithm. Our evaluations in this space are discussed in Sections 2.6.6 and 2.6.7.

Differences in Cryptographic Functions. There are fundamental differences in different classes of cryptographic algorithms. Cryptographic features (see Section 2.2.2) present in one algorithm may not be present in others. For instance, the data avalanche effect [21] which says that an insignificant change in the input parameters can make significant differences in the output, works well for identifying block ciphers. In the case of stream ciphers, there are no such observations like data avalanching. Tools must be designed to account for these differences if they wish to identify broad classes of algorithms.

Compilers. The choice of compiler can significantly impact the binary structure of a program in various ways such as format, linking, inline functions, and compatibility with different architectures. The incorporation of anti-analysis techniques further adds to the complexity of cryptographic function detection. Consequently, working with different compilers can introduce numerous challenges. Our study (Section 2.6.4) demonstrates how a tool's detection capabilities can vary greatly depending on the compiler used.

Comparison to General Binary Analysis. Binary code similarity analysis is an active area of research in the software community. There has been a plethora of research [55–62] to compare two or more binaries to identify their similarities or variances. While such generic binary analysis techniques can be leveraged in the detection of cryptographic functions, future researchers should bear in mind that detecting cryptographic functions in binary code remains a multifaceted challenge. Cryptographic algorithms exhibit significant variability in terms of complexity and implementation. They often involve intricate mathematical operations and employ various techniques such as bit manipulation, modular arithmetic, and logical operations. This complexity can pose difficulties in identifying cryptographic functions without specific knowledge of the algorithm being utilized. In addition, they can be implemented in diverse ways and may not adhere to consistent naming or calling conventions, rendering them more challenging to identify through static analysis alone.

2.8 Conclusion

Cryptographic function identification has been a popular area of study in both academia and industry. However, we noticed a lack of consistent means for evaluation and comparison among tools. As such, in this work we conduct comprehensive reproduction and replication studies on all existing work in the space. To complement the traditional R+R studies, we developed a comprehensive testing and evaluation framework which includes a number of modern cryptographic algorithms and real world examples, that allow for the comparison of both existing and future work in a uniform manner. Finally, we highlight major gaps in existing work, especially as they relate to modern cryptographic primitives and real-world use cases, and discuss a variety of important avenues for future work.

3. SNARKPROBE: AN AUTOMATED SECURITY ANALYSIS FRAMEWORK FOR ZKSNARK IMPLEMENTATIONS

This chapter is based on joint work with Christina Garman (Purdue University) along with Yuquan Xu (Purdue University, Georgia Tech) [1]*. The paper was published in proceedings of the 2024 International Conference on Applied Cryptography and Network Security, pages 340-372 [63].*

3.1 Introduction

We have seen a growing interest in privacy and privacy-enhancing technologies from the general public [64], which has led to subsequent increased interest from parties that build and deploy the technologies that we use every day, with even the US government expressing interest in ways to best deploy privacy-enhancing technologies for data analytics [65]. One of the key components in many privacy-enhancing protocols are zero-knowledge proofs [66], which are cryptographic algorithms that allow one party (the prover) to prove to another (the verifier) that a statement is true, without revealing any information beyond the validity of the statement itself. Of these, one of the most popular instantiations are zkSNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) [67–69], because of their small proof size, fast verification, and expressiveness.

Because of this popularity, we have seen an explosion of new zkSNARK protocols and libraries being developed in academia, and substantial interest from industry and other domains in actually deploying these protocols for real world usage [70–74].

Unfortunately, they can be quite difficult to implement correctly, as the protocols themselves can be complicated and involved, and much of the work to generate a single proof for a single application is often done manually and hand-tuned to ensure maximum performance, with developers often working at the circuit or gate level to design the best protocols. Additionally, as we have seen in the past, deploying complex cryptography has not come without its challenges [75–78], and applications that use zkSNARKs have not been immune to these either, as Zcash for example has found various cryptographic bugs in different com-

[1]↑This work was supported by NSF grant CNS-2047991.

ponents [79–81]. And checking any cryptographic implementation manually can be time consuming and potentially error prone.

While automated techniques like fuzzing have been used to test cryptographic implementations specifically before [82], we cannot use existing cryptographic fuzzers here as they generally test only for known weaknesses in certain schemes and are unable to produce the types of inputs that we need in a cost-effective way. Additionally, most general purpose fuzzing tools cannot detect what we refer to as *cryptographic logic errors*, i.e., errors that might result in an incorrect computation but not a program crash, particularly with regards to zkSNARK libraries.

All of this motivates us to ask the question:

Can we develop better tooling to automatically check the security of and precisely locate software bugs and cryptographic logic errors in the proof generation processes and libraries of zkSNARK protocols and the applications that use them?

To help answer this question in the affirmative, we design and build `SNARKProbe` to automatically both check the correctness and consistency of proof programs generated by R1CS-based zkSNARK libraries, as well as inspect the security of the libraries themselves and flag any inconsistencies between a protocol's implementation and its description, no matter how minor[2]. Our primary goal is to make the process as automated as possible, thus reducing the chance of human error in the process, as well as lowering the barrier to entry for usage. We wish to enable even those who might not be cryptographic experts to inspect a library or proof implementation without understanding details of the underlying protocol or specifics of the library. Users need only indicate some configuration settings such as the fuzzer parameters and expected proof statement, and our tool will automatically analyze the library and proof program to detect possible implementation and cryptography related errors.

In order to achieve this, we utilize a combination of techniques. Dynamic analysis allows us to trace real-time data and variable values, as well as handle a variety of different zkSNARK libraries written in different programming languages without needing additional

[2]↑Note that this is something that professional security audits do flag, as these can lead to potential future problems, see [81].

(manual) language specific adaptations. Custom fuzzing techniques allow us to produce a variety of valid or invalid R1CS matrices (i.e., proofs) to exercise the different codepaths in a library. SMT (Satisfiability Modulo Theory) solvers allow us to help verify the consistency of a user-specified set of statement equations (i.e., the desired proof) with the actual R1CS matrix used in the application. And the notion of ideal files and a value checker helps ensure that a library correctly realizes the given underlying protocol.

While one could tackle some of this from a formal analysis approach [83], we view `SNARKProbe` to be complementary, as formal analysis works best for specific, static protocol implementations (like libraries), and we wish to also be able to check application-specific proofs. Formal verification in such a scenario would need to be done for *each individual proof*, which is both time-consuming and requires expertise in such techniques for every project that wishes to use a zkSNARK. `SNARKProbe`, on the other hand, is designed to work with a variety of different protocols and to be easy to run on a given proof implementation in an application, without requiring substantial expertise. We will see later on how a formally verified library for a specific protocol could be adapted as part of `SNARKProbe` to provide stronger guarantees, though we leave this for future work. As such, we believe `SNARKProbe` is a step in the right direction for ensuring the correctness of existing zkSNARK implementations and applications.

We choose to focus on two widely used zkSNARK protocols, Pinocchio [68, 84] and Groth16 [69], and four libraries, `libsnark` [85], `Bellman` [86], `arkworks` [87], and `gnark` [88]. `libsnark` implements both Pinocchio and Groth16 in C++, while `Bellman` and `arkworks` implement Groth16 in Rust, and `gnark` implements Groth16 in Go. These represent four of the most popular and widespread open source libraries that have seen real world usage and underpin many projects that use zkSNARKs. This selection also demonstrates the flexibility of `SNARKProbe` to handle different R1CS-based zkSNARK protocols, as well as diverse languages.

In addition to designing and implementing `SNARKProbe`, we tested its performance extensively, using a number of different statement types to test the Constraint Checker, and a wide variety of R1CS matrix sizes and configurations (which are representative of statements of varying complexity) to test SnarkFuzzer.

We demonstrate performance that we believe is both reasonable and scalable for existing zkSNARK use cases[3]. We also evaluated `SNARKProbe`'s ability to detect potential inconsistencies or different types of errors. While we did not find any new exploitable errors, we did find a number of issues across the various libraries that we consider to be potentially unsafe or that might require special care by a developer to navigate correctly.

3.1.1 Our contributions

In this paper we make the following contributions:

- We design `SNARKProbe`, a tool that can automatically check for potential software errors and cryptographic logic errors, as well as for consistency in the implemented proof statement, in R1CS-based zkSNARK libraries.
- We implement and build `SNARKProbe`[4] for two zkSNARK protocols, Pinocchio [68, 84] and Groth16 [69], and four libraries, `libsnark` [85], `Bellman` [86], `arkworks` [87], and `gnark` [88], to demonstrate its flexibility and extensibility.
- To the best of our knowledge, we are the first to explore applying fuzzing and other analysis techniques to zkSNARKs specifically.
- We evaluate the performance of `SNARKProbe` in an extensive set of experiments, demonstrating its acceptable performance for real world applications.
- We demonstrate and discuss `SNARKProbe`'s ability to *automatically* catch different types of errors in zkSNARK libraries, finding seven inconsistencies and successfully locating a prior CVE.

3.1.2 Chapter Organization

In Section 3.2, we discuss the necessary background and related work. In Section 3.3, we provide a brief overview of `SNARKProbe`, as well as its intended use cases. In Section 3.4, we introduce the high level design and goals for each component of our tool with the concrete tools and techniques that allow us to realize these design goals. In Section 3.5, we evaluate

[3]↑And, in fact, in one instance the underlying library was actually unable to scale and run before we ran into any issues with our tool.
[4]↑https://github.com/BARC-Purdue/SNARKProbe

the performance of `SNARKProbe`, and in Section 3.6 we discuss its ability to detect known errors. Finally, we conclude in Section 3.7.

Possible ethical considerations. While our tool did find a few inconsistencies in gadgets and protocol specification versus implementation throughout our testing, all of these issues were either not exploitable, already known, or just require care from a developer when implementing a proof, and as such we do not believe there is anything to disclose.

3.2 Background and Related Work

In this section, we introduce the necessary background and prior work related to zk-SNARKs, fuzzing, SMT solvers, and automated testing for cryptographic vulnerabilities.

3.2.1 Background

zkSNARKs. In a zero-knowledge protocol [66] a user (the prover) proves a statement to another party (the verifier) without revealing anything about the statement other than that it is true. zkSNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) [67–69] are one of the current most popular zero-knowledge proof systems, and have begun to see increasingly complex, real world deployment in a number of privacy-preserving applications [70–74]. At a very high level, zkSNARKs work as follows. The first step in proof creation is to turn the statement that one wishes to prove into an equivalent form that relies on knowing a solution to some algebraic equations. This representation is then broken down into an arithmetic circuit. To ensure the correct evaluation of the circuit (and thus the proof), a programmer must express a series of constraints on the wires, called a constraint system, which is typically a Rank 1 constraint system or R1CS. zkSNARK libraries often provide an abstraction called a *gadget*, as expressing large programs with constraints can be quite difficult. A gadget allows programmers to specify a series of inputs, hidden internal variables and constraints on the inputs and internal variables. Rather than expressing a large program as many constraints, one can instead express it as some gadgets, and a few constraints binding them together.

Fuzzing. Fuzzing is an automated software security testing technique to explore software for bugs that cause incorrect or unexpected results, program crashes, or in some extreme cases, lead to exploit paths. Fuzzing works by generating inputs and monitoring the program for crashing, assertions, and memory leaks [89] without the need for manual review by developers, security engineers, and auditors, which can be ineffective, expensive, and hard to guarantee complete coverage. An important part of fuzzing is code coverage, a metric used to evaluate how much of the target program is tested. Early fuzzing systems only generated inputs by mutating a seed input and watching for crashes or errors. Over time, fuzzing test structure has been improved and other techniques have been developed to improve the effectiveness, such as coverage feedback [90, 91], advanced algorithms [91–93], and machine learning [94]. Some feedback-guided fuzzing tools have also been developed [95, 96], which have found security vulnerabilities in widely deployed software [97].

SMT Solvers. Satisfiability modulo theory (SMT) is a complex version of the boolean satisfiability problem (SAT) that can determine if a mathematical equation is satisfiable or unsatisfiable. Z3 (which we choose to use in this paper) is an efficient SMT solver that has been used in various software verification and analysis applications [98]. Typically, SMT solvers such as Z3 can only handle first-order logic, but recent research [99] proposed a pragmatic extension for SMT solvers to support higher-order logic. Most SMT solvers are built based on an efficient SAT solver [100], and as SAT is an NP-Complete problem, SMT solving is considered to be an NP-Hard and undecidable problem.

3.2.2 Related Works

There has been work to automate the conversion between proof statement and R1CS [101–104], but they do not necessarily guarantee the correctness of the translation. Manually writing/editing R1CS is also still popular as it can result in substantial efficiency gains. Such work can help secure new applications but cannot help with the security of existing implementations.

Fuzzing has been used for some cryptographic implementations. Special-purpose fuzzing has been used for TLS [105, 106]. CDF [82] targets cryptographic algorithms such as DSA,

Fuzzer	Adapt to zkSNARK	Error Types			Error Location	
		Program	Logic		Code	Scheme
			Crypto	zkSNARK		
AFL [95]	●	●	○	○	●	○
CDF [82]	○	●	◐	○	○	○
SHA-3 Test [107]	○	●	●	○	○	○
TLS-Attacker [105]	○	●	◐	○	●	◐
`SNARKProbe`	●	●	●	●	●	●

Table 3.1. Comparison of `SNARKProbe` and a selection of cryptographic and generic fuzzing tools. A ◐ means achieves in select instances.

ECDSA, and RSA. We are inspired in part by these ideas, though we have no known test vectors or reference implementations to compare against and use more targetted input generation techniques. In addition, most general purpose fuzzing tools cannot detect *cryptographic logic errors* since they are specifically designed to find and trigger standard software errors. We consider a cryptographic logic error to be one that causes a deviation from the expected protocol flow or an incorrect output, without causing an error message or crash. To the best of our knowledge, we are the first to build such fuzzing techniques for zkSNARKs.

We provide a comparison of `SNARKProbe` and the most relevant existing fuzzing tools in Table 3.1, focusing on the types of errors they are able to catch, if they can precisely locate such errors, and their adaptability to zkSNARKs. Currently, most fuzzing tools are limited to identifying software errors. While some tools such as CDF and SHA-3 Test have the capability to identify cryptographic logic errors, their functionality is restricted and cannot be easily extended. Presently, our SnarkFuzzer stands as the only fuzzing tool capable of detecting both software and cryptographic logic errors for zkSNARKs, accompanied by source code references and protocol location information.

Additionally, an important part of many fuzzers is how they handle code coverage analysis. While off-the-shelf tools like LLVM might seem to fit well for this, as it natively provides a source code line coverage data report [108], this is insufficient for our desired application. Our fuzzer focuses specifically on branch coverage and coverage of critical cryptographic components, not generic line coverage. Additionally, we found that many existing

zkSNARK libraries may not work well (or even compile) with LLVM. This motivates our development of the Branch Model and its use of GDB, as existing tools do not suffice.

There has also been prior work on automating the development of cryptographic attacks. Much of this work has focused on side-channels [109–111], though it has also seen other applications [112]. Prior work has also sought to automatically deploy existing, known attacks [113]. These are largely orthogonal and do not extend to existing zkSNARK systems. Such attack models work to model timing results by combining a fixed secret input with many other inputs that are chosen by an attacker and often employ SAT solvers. Recent work has provided a more general framework for such techniques and extended them to automate format oracle attacks [112]. This also extends work which sought to automatically deploy existing, known attacks [113]. While these works also seek to automatically detect vulnerabilities in different contexts, and use some similar techniques such as SMT solvers, these are largely orthogonal to our current work and do not extend to existing zkSNARK systems.

There is a growing trend towards computer aided cryptography and formal verification of implementations (see [114] for a detailed discussion). Project Everest [83] contains a variety of formally verified software components. While we are seeing increasingly complex protocols being formally verified [115], we have not yet seen these techniques applied to zkSNARKs. We see our work both as a step towards a formally verified zkSNARK implementation, bridging the current gap through the use of heuristic techniques like fuzzing, as well as a complement to it. A formally verified implementation could be used to provide a trustworthy ground truth for our fuzzing techniques, for example. In addition, formal verification is typically targeted at a single implementation, which necessitates re-running the often complicated process for each new application. `SNARKProbe` is designed to streamline the process and make it easy for a user to check a variety of applications or libraries, with little manual effort.

3.2.3 Challenges and motivation

Based on the prior work and current techquies, there are following challenges to evaluate the security of a zkSNARK library, which are also our motivation to develop our `SNARKProbe`.

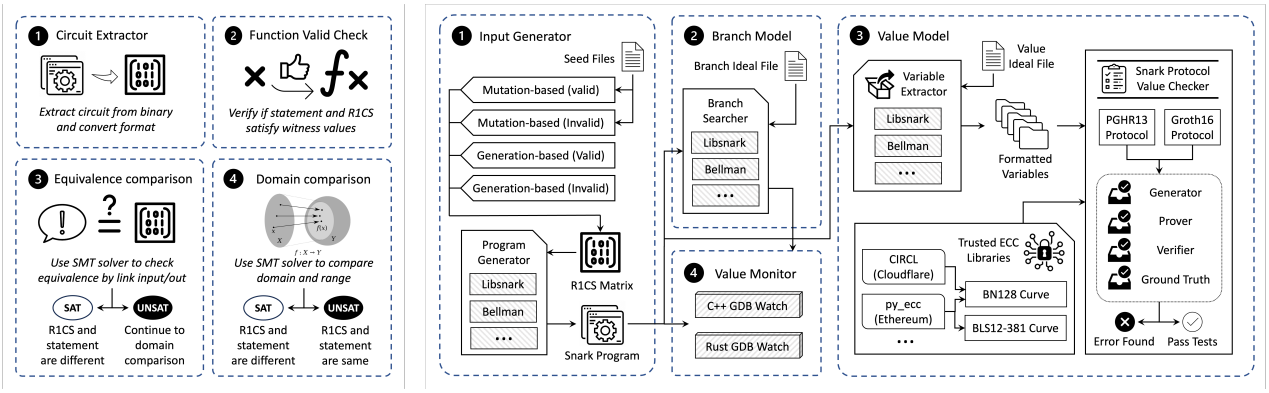
- Manually evaluating the security of cryptographic library source code requires a significant investment of time and resources.
- Existing cryptographic fuzzers and their techniques does not support zkSNARK or other zero knowledge proof protocol due to their complexity.
- General-purpose fuzzing tools cannot detect *cryptographic logic errors* and are unable to produce suitable input for zkSNARK programs.

3.3 Overview

Before diving into the details of our design, we give a brief high-level overview of our `SNARKProbe` system, its workflow, and the intended use cases and users.

3.3.1 Users and Use Cases

While many long-standing zkSNARK libraries, such as `libsnark` and `Bellman`, have been widely used and tested by researchers and security engineers, there are continuously new libraries being developed in various programming languages, new applications utilizing existing libraries for different purposes, and new zkSNARK cryptographic protocols being created. Auditing a new cryptographic library or piece of software that utilizes complex cryptography takes a significant amount of time, resources, and expertise, and many individual developers or small organizations might not have the ability or means to do so. In addition, users of such systems might also wish to convince themselves that these components are secure and correct. `SNARKProbe` works to allow its users to verify just this in a manner that is low cost and accessible for non-experts that have little prior experience in cryptography. Our tool outputs a full evaluation report that includes a notion of security confidence, any discovered (known) security vulnerabilities or warnings, and potential risks of the target library or proof.



(a) Constraint Checker (b) SnarkFuzzer

Figure 3.1. Workflow of `SNARKProbe`. White boxes represent subcomponents universal for all zkSNARK libraries, forward diagonal stripes represent those designed for a specific library, and backward diagonal stripes represent those designed for a specific programming language.

We view the targeted users of `SNARKProbe` to fall into three main categories: 1. An application user that wishes to evaluate if a given (open-source) application correctly implemented their specified proof statement without having to dig through source code or understand the protocol; 2. Developers who want to build applications using zkSNARKs and wish to gain assurances in the correctness of a library and that a proof they built correctly realizes a given statement; and 3. Security engineers that want to automatically scan a library/application and detect various issues, such as edge case crashing, cryptographic operation errors, and/or inconsistencies with protocol descriptions, allowing them to focus their time and manual effort on potential issues. `SNARKProbe` helps achieve all these use cases by outputting a full evaluation report that includes a notion of security confidence, any discovered (known) security vulnerabilities or warnings, and potential risks of the target library or proof.

3.3.2 Workflow

We now outline the workflow of running `SNARKProbe`, as well as establish terminology, before presenting the details and implementation in subsequent sections. `SNARKProbe` is composed of two broad sub-tools: the Constraint Checker and SnarkFuzzer. These compo-

nents can operate both independently of each other as well as in sequence, depending on what the user wishes to check.

Recall that at a high level, a zkSNARK proof is generated in two main phases. First (*circuit generation*), the user has a *(proof) statement* that they wish to prove about, which requires converting this statement into a *circuit* that the zkSNARK protocol understands. *R1CS* is a specific form of this that we focus on in this work. Second (*proof generation*), a zkSNARK library takes the circuit (R1CS) as input and produces a proving key, verification key, and a *proof*.

The first step verifies the consistency of a user-specified set of proof statement equations with the actual R1CS matrix used in the application, to ensure that the circuit generation process generates the same proof as the developer states. The Constraint Checker uses an SMT solver to compare the input values, output values, domain, and range of the given statement equations with R1CS matrix to find any errors in the conversion. Figure 3.1a shows the internal steps of the Constraint Checker, which we discuss in detail in Section 3.4.1.

The second step checks the correctness of the library and proof generation process. Figure 3.1b shows the steps of SnarkFuzzer, which we discuss in detail in Section 3.4.2. First, the Input Generator produces an R1CS matrix (i.e., proof) using a variety of fuzzing techniques and compiles this into an executable proof program with the selected library, which serves as the input for the Branch Model. Then, the Branch Model investigates the visitation status for each branch during execution and provides feedback to the Input Generator to determine if another input is needed. This coverage feedback fuzzing helps improve the coverage rate of SnarkFuzzer, which improves the chance of catching errors in the target library. Finally, the Value Model re-evaluates the protocol calculations to detect any potential cryptographic logic errors and looks for inconsistencies caused by unexpected value changes or implementation errors.

Together, these two components allow `SNARKProbe` to evaluate the end-to-end correctness and security of a specific implementation of a given proof statement for a given library (or to individually check different parts if desired).

Unlike standard fuzzers, SnarkFuzzer can detect both software and cryptographic logic errors. We consider a *software error* to be a bug or fault in the software that causes un-

intended behaviors or incorrect performance, such as crashing, overflow, or a system error. A *cryptographic logic error*, on the other hand, is one that causes a program to produce an incorrect result due to errors in the cryptographic protocol implementation, such as incorrect mathematical operations or disregarding the specification. A cryptographic logic error will not cause a software crash or raise an error message, making them much harder to detect. For a basic example, in RSA, software errors might cause a program crash, but cryptographic logic errors are issues where the ciphertext value was not computed correctly ($m^{\mathrm{e}} \mod N$ computed incorrectly) or too large a ciphertext was used (e.g., larger than the modulus).

While we currently focus on R1CS-based zkSNARKS, we believe this high level design can be adapted without major changes to support additional (newer) types of proofs, particularly in the case of SnarkFuzzer, which would largely only involve changing the input generation methodology and format.

Why dynamic analysis? Dynamic analysis supports our goal of finding cryptographic logic errors by allowing us to trace and extract real-time data and variable values, which we can then use to check for cryptographic protocol conformance and implementation errors. It also allows us to easily support a number of different zkSNARK libraries written in different languages without the need for additional manual intervention, extensions, or program instrumentation by the user, making `SNARKProbe` flexible as well. Our design choices mean that is easy to extend `SNARKProbe` to support new languages and libraries, without needing to investigate new instrumentation techniques or tools.

3.4 Design and Implementation

In this section, we introduce both the high level goals and concrete implementation decisions, tools, and techniques that allow us to realize `SNARKProbe`. We start with the Constraint Checker and then discuss the various components of SnarkFuzzer. These two sub-tools can be used either in conjunction with each other, or separately, depending on what the developer wishes to check.

3.4.1 Constraint Checker

Goals: *The Constraint Checker helps ensure that the realized proof is equivalent to a specified statement. This also allows the Constraint Checker to find errors in flattening and/or gadget use. It achieves this by comparing the equivalence of a user-specified proof statement and the compiled R1CS gates to ensure that the proof generated by the library represents the developer's specified statement*[5].

Design. The first step in generating a zkSNARK proof is converting the desired proof statement equations into R1CS. These R1CS gates should have the same domain, range, and output value as input statement equations. To check this, we developed our Constraint Checker, which leverages an SMT solver and associated techniques to verify the equivalence of R1CS gates to the specified proof statement functions[6]. To run the tool, the user only needs to write a short script to define the statement function, primary inputs, and auxiliary inputs in their zkSNARK proof, and everything else then happens automatically. Figure 3.1a shows the key steps in the Constraint Checker, and we also provide an example in Appendix B.1.3. We note that our SMT solver could be replaced by any other techniques that allow one to check for function equality and fit the desired goals.

Implementation. The Constraint Checker contains two major components—the Circuit Extractor and a three step equivalence check—which we discuss now. We discuss optimizations in Appendix B.1.1.

Circuit Extractor. The Circuit Extractor can extract R1CS gates with private inputs, witness, and gadget relations from an executing zkSNARK binary program. We do this using GDB. We set a GDB breakpoint at the circuit declaration line, convert the circuit to an R1CS matrix when the program reaches the breakpoint, and save the formatted R1CS matrix for equivalence comparison. Automatic extraction both makes the tool more usable and reduces the potential for human errors in the equivalence comparison. Many zkSNARK libraries use Montgomery modular multiplication as an optimization. We also developed a

[5]↑We note that we cannot do anything about the garbage-in-garbage-out problem (for example, the case where a developer implements a proof that misses a necessary case, such as omitting checking if a value is greater than 0). We can just help determine if the specified statement and realized proof are likely equivalent.
[6]↑We work around the general undecidability of function equality by operating in the more constrained setting (over the finite fields) and adding special rules for primary variables and auxiliary variables.

Montgomery representation translator that can convert the Montgomery form to a standard integer format, so that our circuit extractor can produce a z3py (our SMT solver of choice) readable integer format.

After extracting the circuit, the Constraint Checker uses the SMT solver to verify the equivalence of functions through a three step process: a function valid check, equivalence comparison, and domain comparison. If one or more tests fail, then we determine that the given statement equation is not equivalent to the produced program. Otherwise, then we can provide assurances that it is highly likely that the proof realizes the specified statement correctly[7].

Function Valid Check. As the witness should be a solution in both the statement equations and equations built by the R1CS gates, we first create two SMT solvers S_1 and S_2 to verify that fact (see Appendix B.1.2 (b)). The Constraint Checker considers the statement equations and R1CS gates to not be equivalent if the witness is not a valid solution for both.

Equivalence comparison. The equivalence comparison verifies if the statement equations $y_1 = f(x_1)$ and R1CS gates $y_2 = f(x_2)$ have the same output for all the same inputs. First, the Constraint Checker creates a solver S, and adds $y_1 = f(x_1)$, $y_2 = f(x_2)$, $x1 = x2$ and $y_1 \neq y_2$ to the solver S (see Appendix B.1.2 (c)). If S results in `SAT`, then the SMT solver has discovered that the statement equations and R1CS gates have a different output for at least one input and thus considers them not equivalent, and otherwise, it results in `UNSAT`. Like in many other use cases, the Constraint Checker cannot guarantee equivalency if the SMT solver returns `UNSAT` or `UNKNOWN`. Note that the SMT solver may return `UNKNOWN` for a variety of reasons, including running out of memory, or if the quantifier-free fragment is undecidable (such as nonlinear integer arithmetic) or too "expensive" (such as finding primes $p * q = N$ for some N).

Domain comparison. The SMT solver cannot directly calculate an equation's domain, but it can find the maximum and minimum values for an equation (Figure B.1a in Appendix B.1.4 shows an example of different upper and/or lower bounds). We also use the SMT solver to find out if there is a gap between the domains' of the statement equations and R1CS gates,

[7]↑We cannot *guarantee* equivalence due to the fact that the SMT solver might output `UNKNOWN`, as well as possible domain gaps (see Appendix B.1.4).

though we cannot guarantee to find all such gaps that may exist (see Appendix B.1.4 for a detailed discussion).

3.4.2 SnarkFuzzer

Goals: *SnarkFuzzer is a smart fuzzing tool with the primary goal of finding potential logic or software errors in zkSNARK libraries. It produces zkSNARK programs with different proofs as input, to detect and catch these errors and provide full (important) code coverage.*

SnarkFuzzer is composed of four major subcomponents: the Input Generator, Branch Model, Value Model, and Value Monitor. The Input Generator is responsible for generating an R1CS matrix and converting it to a zkSNARK program as input for the target library. The Value Model and Value Monitor are designed to detect implementation and cryptography related errors. The Branch Model helps manage branch coverage and provides feedback to help SnarkFuzzer decide if it needs to generate another input program. Figure 3.1b shows the structure of SnarkFuzzer with both its high level design as well as its implementation details.

SnarkFuzzer: Input Generator

Goals: *The Input Generator produces valid or invalid R1CS matrices and compiles them into executable proof programs for use by other components as testing inputs. This should be done in a "smart" manner to help better catch errors.*

Design. The first step for our fuzzer is the generation of inputs to test a target library. In SnarkFuzzer, these inputs are simply valid or invalid R1CS matrices that represent a variety of (possibly random) proof statements. To improve the chances of catching potential logic or software errors in the library, we desire this input generator to be "smart", in the sense that it should not just randomly generate inputs, but should take into consideration both the concept of valid/invalid proofs, as well as proof or R1CS edge cases. At a high level, the Input Generator generates an R1CS matrix, and then the Program Generator uses this matrix to produce and compile an executable zkSNARK program with a given library. The compiled program will then be used as input for other components, like the Branch Model, Value

Model, and Value Monitor. Before generating the R1CS matrix, the Input Generator needs to decide on the size of the generated matrix (number of rows n_r and columns n_c) and the number of public inputs n_p. This can be done by simply choosing randomly, or in a smarter way by choosing from a specified distribution. The Input Generator can be instantiated with any type of fuzzer that meets the aforementioned goals, giving it the flexibility and adaptability to use new techniques and tools, as they are developed.

Implementation. We developed four different methods of input generation to produce these matrices based on mutation-based and generation-based fuzzing. These methods are complementary and all work in tandem to generate the fuzzing inputs (the user can specify the percentage of inputs generated by each method). Each method targets a specific type of R1CS matrix to provide better overall coverage and comes with its own set of unique benefits (and trade-offs).

Generation-based invalid matrix. Our generation-based fuzzer is designed to create an R1CS matrix from scratch. Generation-based fuzzing has the advantage of not depending on input seeds, allowing it to function without (user) input. The random number generator will generate $n_r * n_c$ integers as an R1CS matrix and $n_r - 1$ integers as the witness w (since the first value of the witness is always equal to 1), and we use the constraint relation $w \cdot A \times w \cdot B - w \cdot C = 0$ to verify the ground truth of the matrix. This tests a library with possible but unrealistic examples to search for edge case errors that are hard to reach with traditional examples.

Generation-based valid matrix. To produce a valid matrix that simulates a real world proof example, the random number generator will generate $n_r - 1$ integers as a witness w. Then, a valid matrix will be calculated with z3py based on this witness and the constraints relations in $w \cdot A \times w \cdot B - w \cdot C = 0$. Depending on the actual values in the randomly chosen witness, the complexity of solving the equations to fill out the matrix may vary, but a larger matrix (i.e., larger witness) usually results in a longer generation time. As such, we recommend that for testing large R1CS matrices, the generation-based fuzzer is run once to generate an input seed that can be given to the mutation-based fuzzer.

Mutation-based invalid matrix. The mutation-based fuzzer requires an R1CS matrix with a number of public variables as a seed, and then flips values to generate a new matrix. We

use Atheris, a mutation-based, coverage-guided fuzzer developed by Google [116]. The seed is converted into a byte array, then Atheris will flip the array, and then it is converted back to a (new) R1CS matrix. Because we have randomly flipped values in the matrix, the R1CS constraint relations will be destroyed, and the matrix will be invalid. This technique can work off of and mutate input examples that might be hard to produce by generation-based fuzzing.

Mutation-based valid matrix. Our mutation-based fuzzer can also produce a valid R1CS matrix. Instead of flipping random values, a random number c will be generated, and the new matrix M will be calculated based on the seed matrix M_0 as $M = cM_0$. This multiplication will not break the constraint relations, so the new matrix is still valid. This can be used with the generation-based fuzzer to produce valid large R1CS matrices in a short time, thus increasing performance.

SnarkFuzzer: Ideal Files

Goals: *The ideal files act as a guide for the Branch Model and Value Model to check code coverage and protocol calculations. They contain information related to the source zkSNARK library such as branch locations, variable names, and data types.*

Design. Each library and protocol needs two ideal files: an ideal branch file that contains the locations of the branches in a library (automatically generated), and an ideal value file that contains information about the variables in a program (manually generated). Manually creating the ideal value file does not require any specialized knowledge related to zkSNARK protocols, the library, or cryptographic techniques, and the user only needs basic skills such as defining functions, conditions, and assigning variables. These ideal files can be reused for any program using the same protocol and library.

Ideal Branch File. An ideal branch file contains protocol-specific information related to the branches in a library. Function calls, if-conditions, and loops are all considered branches. The branch file also contains information about `IMPORTANT` versus `UNIMPORTANT` branches to help the Value Model. Important branches include cryptographic calculations and other

core protocol components that we wish to ensure testing and coverage of, while unimportant branches are those that do not involve relevant computation.

Ideal Value File. An ideal value file contains location information for all variables that must be used to evaluate the protocol calculation. The Value Model will use line and path information in this file to extract all variables' values. Only variables defined in the official protocol paper should be added to the ideal value file.

Implementation. We have already developed the ideal files for Pinocchio and Groth16 in `libsnark` and Groth16 in `Bellman`, `arkworks`, and `gnark`.

Ideal Branch File. By default, all branches are marked as `IMPORTANT` and thus covered. We asked four researchers in our group (including people with experience using zkSNARKs and those with only programming experience) to label branches as `IMPORTANT` or `UNIMPORTANT`. To ensure SnarkFuzzer does not miss any critical components, only branches labeled as `UNIMPORTANT` by all four researchers could be marked as `UNIMPORTANT` in our example ideal branch files[8]. We provide an example snippet of an ideal model file for `libsnark` in Appendix B.2.1.

In order to find uncallable functions, we used a static analysis tool called Doxygen [117] to draw the library functions' call graph. Doxygen can help a user generate the call graph for a library, and by using this call graph, the user can easily find unused and uncallable functions to reduce fuzzing costs.

Ideal Value File. The ideal value file contains "official" variable names, variable types, and variable names, path, and line number (the final assignment location for a variable) in the library's source code. We provide an example snippet of an ideal value file for the `libsnark` library in Appendix B.2.1. While line numbers might change as code is maintained, one should not need to manually create a new file often. If there are only minor or non-cryptographic related changes, our tool will automatically match the new line numbers.

[8]↑Incorrectly labeling `UNIMPORTANT` branches as `IMPORTANT` only increases the fuzzing time without affecting the accuracy of the analysis.

SnarkFuzzer: Branch Model

Goals: *The Branch Model monitors and records branch visitation status as SnarkFuzzer runs, and informs and stops it after all* `IMPORTANT` *branches have been covered.*

Design. The Branch Model is used to evaluate if the target library has been fully tested. Traditional fuzzers often work by examining utilized memory bytes in a binary or lines of source code hit to check for program coverage, and then mapping this coverage to an input to determine the next one. However, in a zkSNARK library, it is difficult to correlate the exact relation between a specific branch and values in the input program that could trigger it. Therefore, we instead use the idea of function and condition branches to evaluate coverage.

Since any unvisited branches may cause errors, the Branch Model evaluates each input program to acquire the list of branches that the program visited. The Input Generator will continuously generate new program inputs with different proof statements/matrices until all `IMPORTANT` branches have been visited.

To ensure that we achieve both breadth and depth in code coverage, the Branch Model also works to ensure that each branch is covered multiple times with different inputs, keeping a count of the number of times each `IMPORTANT` branch is hit. We allow the user to define a visitation threshold to balance performance with coverage and confidence. As more inputs are generated, we have a higher chance of finding potential issues in the target library or, conversely, more confidence that the target library is safe, at the cost of runtime.

Implementation. Our Branch Model uses GDB breakpoints to investigate the visitation status for each branch defined in the ideal branch file. A GDB breakpoint stops the program whenever the selected location in the program is reached for debugging, and it can be set by line number, function name, or address in the program. When the Input Generator produces a zkSNARK program, the Branch Model sets a GDB breakpoint at the start line for each branch. If a branch is then visited during runtime, GDB will temporarily stop the input program at the corresponding breakpoint. After the program is finished running, the Branch Model records a list of visited branches for the current program. If all important branches have not been visited at least once, or the visitation threshold has not been met, the Branch

Model will instruct the Input Generator to generate a new program with a different proof until all important branches have been covered.

SnarkFuzzer: Value Model

Goals: *The Value Model's goal is to find cryptographic logic errors, such as inconsistencies with the protocol definition or incorrect cryptographic operations.*

Design. The Value Model is one of the most important components in SnarkFuzzer, and one of the biggest improvements compared with other fuzzers. Its job is to detect cryptographic logic errors in a library. At a high level, the Value Model must extract all necessary variable values from the input program, which it then feeds to the other components. It achieves this through two major sub-components: the Value Extractor and Value Checker.

Value Extractor. The Value Extractor extracts variables' values from the source zkSNARK program, which are then used in the Value Checker. The ideal value file provides a list of variables that need to be extracted. Then, the Value Extractor recursively extracts these values from the last assignment location in the program and library, and reformats the values to the Value Checker readable format.

Value Checker. The Value Checker re-evaluates all protocol calculations and compares these ground-truth values to the values extracted from the input program by the Value Extractor to find any possible logic errors. In the re-evaluation process, the Value Checker will faithfully follow the protocol from the formal specification without any optimization, simplification, or reformatting. To build a trustable re-evaluation process, our Value Checker uses ECC libraries that have been tested and widely used. In the future, this component could be replaced by any trusted evaluation source, such as a formally verified implementation.

Implementation.

GDB PrettyPrint. GDB can display both the structure and value(s) for a variable. However, for more complicated variables GDB displays some unnecessary variable structure and data. Also, different libraries usually have different data structures (even for the "same" variables), and, as such, the Value Checker cannot directly use the raw, unformatted variable value extracted from the library. For example, Figures B.4a, B.4b, and B.4c in Appendix B.2.2

show a G_1 type variable that represents the same number in both `libsnark`, `Bellman`, and `arkworks` but has different structure and value when printed naively.

To make the Value Checker more universal, we developed a PrettyPrint functionality for each library to convert all data structures into the same format. The goals of PrettyPrint are twofold: 1) to allow the extractor to keep only the necessary values for a variable, and 2) to convert the same types of variables from different libraries into a common universal format that the Value Checker can then accept. Figure B.4d in Appendix B.2.2 shows an example of the PrettyPrint result for the same G_1 variable as before.

Value Extractor. The Value Extractor extracts all variables defined in the ideal value file using GDB and PrettyPrint. As the ideal value file contains the line number where each variable is assigned, the Value Extractor can automatically set up a breakpoint for each variable immediately after its assignment. Then, when the input program reaches this, the Value Extractor uses PrettyPrint to save the variable's formatted value.

Value Checker. We developed the Value Checker for Pinocchio by following [68] and [84] and Groth16 by [69] and [118]. To help ensure accurate re-evaluation of the protocol calculations, the Value Checker uses exactly the same variable names as the original papers, and we follow the protocol description exactly, without any optimizations, to avoid introducing mistakes.

ECC Libraries. We use two well-tested elliptic curve libraries to demonstrate diversity and flexibility. py_ecc [119], developed by Ethereum in Python, supports both the BN128 and BLS12-381 curves. CIRCL [120], developed by Cloudflare in Go, supports BLS12-381. Since it is written in Go, we developed the necessary APIs so that our Python-based Value Checker can access the functions. If users would like to test a new zkSNARK library using a curve that py_ecc or CIRCL do not support, they can easily plug it into the Value Checker in this same manner.

SnarkFuzzer: Value Monitor

Goals: *The Value Monitor detects any unexpected value changes after the extraction of initial variable values from the source program.*

Design. While the Value Model extracts all necessary variables' values at the location where the variables are first assigned, and we also need to check if there is a value change after the model extracts these values to ensure consistency between the library and the Value Checker. The Value Monitor can also detect any unexpected side effects caused by the library or dependent functions due to incorrect implementation, out of memory, or other unexpected errors.

Implementation. GDB watch allows SnarkFuzzer to monitor value changes in a debugging session with four CPU debug registers called hardware watchpoints. A hardware watchpoint is very efficient, but it does not support larger data structures given the limited debug registers. Software watchpoints will be automatically applied if GDB tries to setup a watchpoint for a variable that cannot be handled by a hardware watchpoint. Unlike hardware watchpoints, software watchpoints are extremely slow, and watching dozens of variables may require unacceptable processing time. For example, using GDB software watch for a single variable `A_query` in `libsnark` takes more than 30 minutes.

Therefore, we instead used Valgrind, a dynamic analysis tool that analyzes a program to automatically detect memory management and threading bugs. Valgrind has a gdbserver to simulate traditional GDB hardware watchpoints. Simulation still takes longer than pure GDB hardware watchpoints, but the hardware watchpoint simulation is much faster than GDB software watchpoints. Meanwhile, Valgrind hardware watchpoint simulation does not have any limitation on the number and length of variables, so Valgrind can be used for any zkSNARK source program and libraries with complex data structures.

3.5 Performance Evaluation

Our first set of evaluations for `SNARKProbe` explores its performance and scalability. All experiments run on an Intel Core i7-8700 CPU @ 3.20 GHz $\times$ 12 with 32 GB of RAM running 64-bit Ubuntu 22.04 LTS.

Statement	Equation	Runtime (in seconds)	
		SMT Solver	Overall
Cube	$y = x^3 + x + 5$	0.49	4.18
Cube with Error	$y = x^3 + x + 5$	0.24	3.94
Comparison	$x < 60$	0.54	12.39
SHA-256 Hash	$y = \texttt{sha256}(x)$	0.71	15.39
Inner Product	$\langle u, v \rangle < c_1 \wedge \langle u, v \rangle > c_2$	0.59	18.34

Table 3.2. Constraint Checker runtime for different proof statements

3.5.1 Performance of the Constraint Checker

We start by testing the Constraint Checker and evaluate its performance on statements with different types of operations and complexities, and present the runtimes in Table 3.2. There are two runtime results for each experiment. We first recorded the runtime for the SMT solver, which is the most important step in the Constraint Checker, and we also recorded the total runtime, including the processing time of the Circuit Extractor, SMT solver, and other internal evaluations.

We tested different types of proofs, including the well-known cube example, a comparison (which requires using a gadget), a hash (a complicated mathematics equation), and a proof with inner product, logical operators, and comparison (which requires multiple gadgets and more complex logic). As expected, as we progress in terms of "complexity", runtime generally increases. Interestingly, the hash proof takes the longest in the SMT solver, demonstrating that there are different notions of "complexity" than just statement size.

We also sought to test if the correctness of the statement checked had any impact on performance. We did this by running two different experiments with the cube proof, one where we model a developer correctly flattening the statement equation into R1CS, and a second where we model a mistake in the flattening that introduces an error. Our Constraint Checker took a shorter time to evaluate the incorrect cube program since the Constraint Checker aborts as soon as it finds an error (such as a different domain or output value).

3.5.2 Performance of SnarkFuzzer

We tested SnarkFuzzer on `libsnark`, `Bellman`, `arkworks`, and `gnark` to evaluate the processing time and performance in different configurations. All experiments run on a single core, except for the Value Monitor which is trivially parallelizable across all cores. We start by evaluating the overall tool to examine the runtime percentage that is dedicated to each sub-component. We then drill down and individually explore the scalability of each sub-component to gain a fuller understanding of SnarkFuzzer's overall scalability.

Overall Runtime with Different Libraries and Protocols. We first explore the proportion of total running time that each different facet of SnarkFuzzer takes, and present the results in Figure 3.2.

We ran with five different configurations: `libsnark` with Pinocchio, `libsnark` with Groth16, `Bellman` with Groth16, `arkworks` with Groth16, and `gnark` with Groth16. All of these experiments use generation-based fuzzers to produce valid R1CS matrices (the slowest of the four fuzzing methods we provide) with dimensions in $[30, 60]$. We ran 30 iterations of the Branch Model (since as shown in Figure 3.3 that provides reasonable coverage) and calculated the average runtime to complete a full "run"[9] and proof check for each library and protocol.

We observe that for all libraries and protocols, the majority of time is spent in the Value Monitor and Value Model phases. This is somewhat expected, as the cryptographic operations in the Value Model require time to calculate, and Valgrind's simulation hardware watch also introduces substantial overhead. Since Groth16 has fewer elliptic curve and pairing operations than Pinocchio, testing that involves Groth16 costs less time. However, Valgrind spends a longer time executing a program in Rust than a program in C and Go. Since we use Valgrind in the Value Monitor to find and monitor value changes, we see a longer overall runtime with `Bellman` and `arkworks` than `libsnark` and `gnark`.

We do use small R1CS matrices in these experiments, as the primary goal is to demonstrate the runtime proportion for each component. We make a few key observations about

[9]↑We define a "full run" to be using the Input Generator to produce one zkSNARK program, analyzing coverage status with the Branch Model, and looking for any software and cryptographic related bugs with the Value Model and Value Monitor.

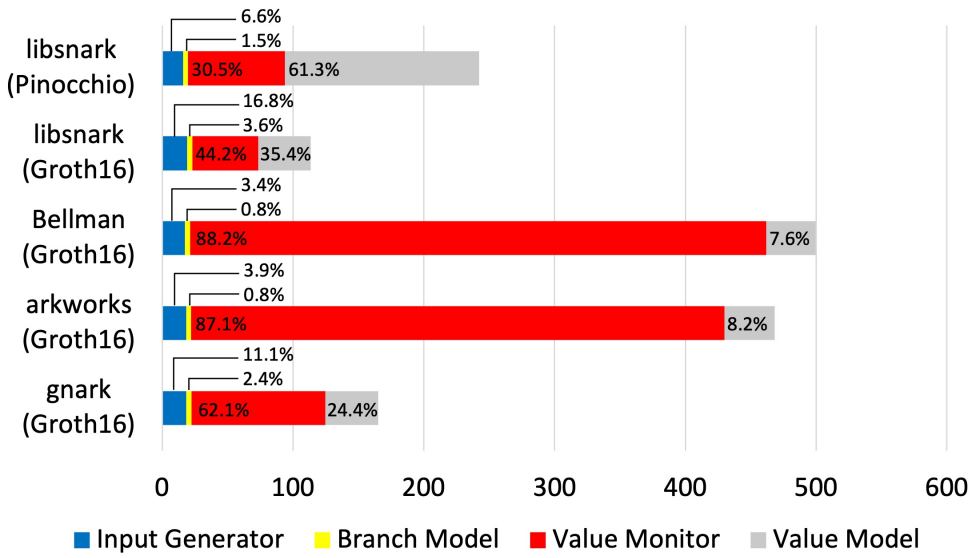


Figure 3.2. Runtime of each component in SnarkFuzzer with different libraries and protocols, both in terms of actual time as well as percentage of the total.

this. We noticed that unless one uses only generation-based fuzzing to produce valid inputs, the size of the R1CS matrix (i.e., the proof) will not substantially affect the performance of the Input Generator. The Branch Model does not work with the R1CS matrix, so different sizes will result in the same runtime. However, a large matrix will significantly increase the runtime of the Value Model and Value Monitor (see Table 3.4 for more realistic values). For total runtime reference, an R1CS matrix of size 100×5000 takes about 940 seconds to run the complete SnarkFuzzer with our generation-based invalid matrix fuzzer.

Performance of the Input Generator. Unlike a mutation-based or generation-based fuzzer for an invalid matrix (which are very quick since they only require straightforward operations), generating a valid R1CS matrix with our generation-based fuzzer requires the SMT solver to produce a matrix based on the random witness array. Thus the processing time depends on the matrix size and random value(s) in the witness list, which generally results in a larger matrix taking longer to process. To validate this, we used our various fuzzers to produce matrices of varying sizes (remember that the rows represent the number of constraints and columns the number of variables). In Table 3.3, we present the processing time for each method (generally under one second for most methods), which confirms our

general intuition. We also note that the Input Generator must compile the input matrix (proof) and source code into a binary program, which can require substantial time that is outside of our control.

Matrix Size		5×10	20×40	50×100	50×200
Generation Based	Invalid Matrix	<0.01	<0.01	<0.01	0.02
	Valid Matrix	0.16	2.06	23.02	51.27
Mutation Based	Invalid Matrix	0.38	0.49	0.65	0.77
	Valid Matrix	<0.01	<0.01	<0.01	0.02

Table 3.3. Runtime (in seconds) of the Input Generator to produce R1CS matrices of different sizes.

Performance of the Branch Model. Next, we explore the coverage ability of the Branch Model in terms of breadth, as well as how quickly we achieve a high percentage of branch coverage. We ran 50 iterations of the Branch Model with `libsnark`, which covers more than 85% of all branches. We stopped our experiments after 50 iterations as SnarkFuzzer did not reach any new branches over the previous 10 prior iterations. Even with this, we note that after roughly 30 iterations we see the breadth of our coverage begin to taper off. A graph can be found in Figure 3.3.

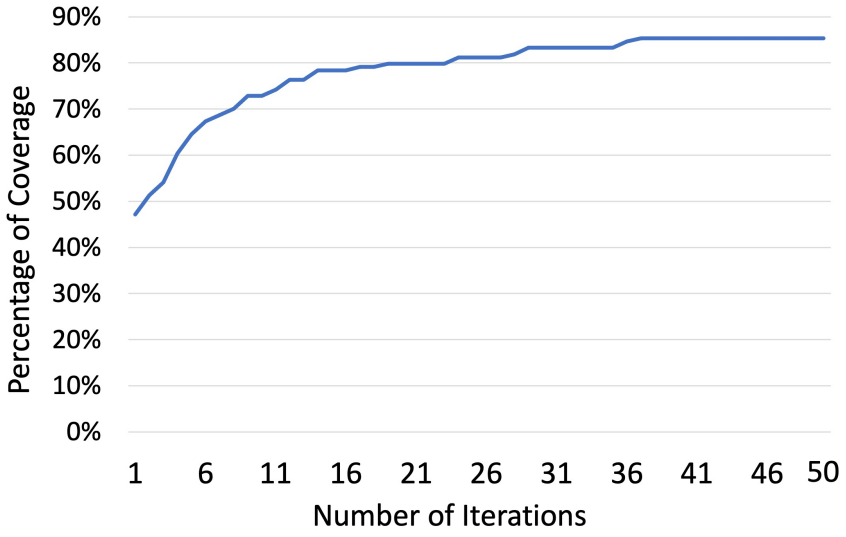


Figure 3.3. Percentage (breadth) of library coverage after a given number of iterations

Performance of Value Model and Value Monitor. The primary effects on overall performance are our Value Model and Value Monitor. Table 3.4 shows the runtime with different matrix sizes. We expect the size of our matrices to be one of the primary influences on the runtime of these components.

Recall that the computation of the proving key requires calculations with the circuit. Therefore, a larger R1CS matrix will result in longer runtime in our Value Model. At the same time, the shape of the matrix will also affect its runtime. As we can see in Table 3.4, a matrix of size 20×45 takes less time than one of size 30×30 or 45×20, even though they all contain 900 integers. In fact, the number of variables (columns) plays a greater role than the number of constraints (rows) in Value Model runtime, since this plays a larger role in the size of the proving and verification keys.

Similarly, the Value Monitor requires more time to watch variables in a program with a larger matrix. However, the reason for this is different. Since the runtime of Valgrind depends on the space complexity of a variable, the matrix shape will not have an effect. Instead, the Value Monitor spends more time monitoring a program with a larger matrix, but it takes similar time for programs with the same number of integers but different shapes. For example, each matrix of size 20×45, 30×30, and 45×20 has 900 integers, and the Value Monitor takes around 46 seconds to run, even though they have a different shape.

Discussion of Total Runtime. Our experiments only provide an upper bound on the worst case runtime for SnarkFuzzer. We did not perform any runtime optimizations, and in all of our experiments, we used the slowest options or configurations that we knew of. While SnarkFuzzer does require extra runtime in the Value Monitor and Value Checker for each individual input seed, in comparison to a traditional fuzzer, which might spend hundreds or thousands of core hours to produce inputs, the framework and Branch Model designs allowSnarkFuzzer to find issues with a relatively small number of generated input seeds. In fact, most errors that SnarkFuzzer detected (see Section 3.6) were found with less than ten seeds. Therefore, we view performance to be acceptable for real world usage.

Matrix Setting		Processing Time (in second)	
Matrix Size	Items in Matrix	Value Model	Value Monitor
10×30	100	23.33	29.21
10×30	300	25.96	33.09
30×10		32.78	34.21
20×45	900	29.52	43.99
30×30		33.01	46.30
45×20		40.14	47.68
45×60	2700	41.29	81.69
60×45		46.03	80.21
100×1500	150000	200.84	223.49
100×5000	500000	580.97	344.83

Table 3.4. Runtime of the Value Model and Value Monitor to process an R1CS matrix of varying size. For example, 10×30 is a matrix with 10 variables and 30 constraints.

3.6 Error Catching Evaluation

Our second set of evaluations demonstrates `SNARKProbe`'s ability to *automatically* catch different types of errors in zkSNARK libraries. We test `SNARKProbe` on `libsnark`, `Bellman`, `arkworks`, and `playsnark`, as well as on the circuit generator `Circom`. Additionally, we show how `SNARKProbe` would have caught a previous CVE in the zkSNARK component of Zcash, a popular privacy-preserving cryptocurrency. In total, `SNARKProbe` detected seven inconsistencies in current zkSNARK libraries, successfully *automatically* located the previous vulnerability, and could potentially detect two additional types of errors. We summarize our results in Table 3.5.

3.6.1 Potentially Locatable in Current Libraries

Incorrect Flattening. There is always a possibility that a developer may generate an incorrect R1CS matrix for their circuit, since this process is often done by hand. While we did not find any examples of this in the few applications we tested, consider the following scenario. Take a statement equation like $x^3 + x + 5 = y$ which the developer will typically flatten into `R1CS_gates` with private input $x = 3$ and public input $y = 35$. This set of

Error/Vulnerability	Type	Affected Component	Program	Found by
Potentially Locatable in Current Libraries				
Incorrect manual flattening by developer	Logic Error	R1CS Circuit	All libraries	ConstraintChecker
Multiple gadget misuse	Logic Error	R1CS Circuit	All libraries	ConstraintChecker
Found in Current Libraries				
Incorrect bit/comparison gadget implementation	Logic Error	R1CS Circuit	libsnark	ConstraintChecker
Mismatch in circuit generation and usage	Logic Error	R1CS Circuit	All libraries	ConstraintChecker
Groth16 pre-pairing computation in Setup	Logic Error	Groth16 Protocol	bellman/arkworks	SnarkFuzzer
Inconsistent QAP extension index usage	Logic Error	PGHR13 Protocol	libsnark	SnarkFuzzer
Toxic waste not safely destroyed	Logic Error	PGHR13 Protocol	playsnark	SnarkFuzzer
Program out of memory with large circuit	Software Error	-	libsnark	SnarkFuzzer
Found in Previous Versions of Libraries				
CVE-2019-7167	Logic Error	PGHR13 Protocol	libsnark (2018)	SnarkFuzzer

Table 3.5. Summary of the current potential errors and inconsistencies, and previous vulnerabilities, that `SNARKProbe` is able to catch and locate.

equations is then equal to $x^3 + x + 5 = 35$. However, the developer *may* incorrectly flatten the statement equation into `R1CS_gates_2`.

$$\texttt{R1CS_gates} = \begin{cases} x * x = w_1 \\ w_1 * x = w_2 \\ w_2 + x = w_3 \\ w_3 + 5 = y \end{cases} \qquad \texttt{R1CS_gates_2} := \begin{cases} x * x = w_1 \\ w_1 * x = w_2 \\ w_2 + 2 * x = w_3 \\ w_3 + 2 = y \end{cases}$$

While `R1CS_gates_2` still satisfies the private input $x = 3$ and public input $y = 35$, causing the underlying library to output an acceptable proof, this flattening does not in fact equal the statement equation $x^3 + x + 5 = y$. Although the statement equation and `R1CS_gates_2` satisfy the witness, and their domains have the same upper and lower bounds, they have a different output range, which our Constraint Checker will catch and flag as an error.

Combining Multiple Gadgets. Recall that a gadget is an abstraction that many zk-SNARK libraries contain to make developing proofs easier for developers. Multiple gadgets may also be combined to create a single proof. Gadgets are generally created and provided independently, which means that one must combine them with care. For example, say a developer uses the gadgets *greater_than* and e*qual_to* to create a proof that a prover has a value which is greater than or equal to a certain number. For this proof, only one input should be used for both gadgets. However, a naive developer may create a proof that has an individual input for each gadget instead of a single input for the entire circuit. In this case, a malicious prover can create a separate program with the same circuit and use two inputs with different values to satisfy each gadget and hence the entire circuit (when in reality these two inputs need to be identical). In this case, our Constraint Checker can detect the inconsistency between the R1CS matrix and statement equations to find potential issues with using multiple gadgets incorrectly.

3.6.2 Found in Current Libraries

Defective Gadgets. Gadgets may be implemented incorrectly, causing inconsistencies between the original (desired) statement equation and the R1CS gates generated. Our

Constraint Checker is able to find these inconsistencies in its checks for equivalence, regardless of whether a gadget was used or not.

The `comparison_gadget` in `libsnark` uses bit shifting to compare two variables A and B by calculating $2^n + B - A$ and then counting the number of 0s and 1s in the resulting bit array (the corresponding R1CS constraints are shown in Appendix B.3). If the developer incorrectly specifies a bit length smaller than the size of A or B, the `comparison_gadget` will not generate the correct R1CS, which may result in a "fake proof"[10]. `libsnark` will not block this behavior or raise any warning messages. Consider a developer that uses the `comparison_gadget` to generate the statement equation $x < 60$ where $x \in \mathbb{F}_p$. The R1CS should then only have a valid solution in $[0, 60)$. However, if an incorrect bit length is provided, we found that all numbers in the range $(21888242871839275222246405745257275088548364400416034343698204186575808494653, p)$ still satisfy this incorrect R1CS, allowing a dishonest prover to generate a valid proof without knowledge of a correct secret value. After our tool flagged this potential issue, on further manual investigation we did see that the `libsnark` source code contains a comment about using the correct bit size, though this could go unnoticed by a novice user or be abused by a malicious one to forge proofs.

Mismatch Between Circuit Generation and Usage. In addition to end-to-end libraries, there are a growing number of circuit generators available to assist developers in constructing R1CS/circuits for proof generation, with `Circom` [104] being a widespread example[11]. `Circom` generates R1CS files that can then be utilized by libraries such as `snarkjs` [121], `wasmsnark` [122], and `rapidSnark` [123]. However, during our evaluation process, our Constraint Checker identified a crucial potential issue: the absence of attribute cross-checking between the circuit generator and formal proof generation library.

While the circuit generator may produce valid R1CS that accurately represents the prover's original statement, the proof generation library might, for example, employ a different elliptic curve and finite field to generate the proof by default. Consequently, when operating under distinct elliptic curve and finite field settings, the circuit no longer main-

[10]↑One that will verify even though the prover does not actually have a valid witness.

[11]↑We note that `Circom` is just an example that we tested, and that the same should hold true for other circuit generators as well.

tains equivalence with the original statement. This discrepancy can be problematic both for a developer and a verifier who are unaware that a circuit can yield different representations under different elliptic curve configurations. In such cases, the prover can produce a seemingly "valid" proof for the verifier without knowledge of the secret value.

Groth16 Pre-Pairing. The Groth16 protocol produces $P = g^{\alpha}$ and $Q = h^{\beta}$ as part of Setup, which are subsequently used in Verify to calculate the pairing $\mathrm{e}(P, Q)$. However, our Value Checker found that `Bellman` stores the pairing $\mathrm{e}(P, Q)$ as part of the verification key instead of P and Q. Pre-computing and directly providing $\mathrm{e}(P, Q)$ is an optimization that helps make verification more efficient (as pairings are quite slow to calculate). As this is a well-known optimization, we can confidently say that this will not cause any sort of vulnerability, but we err on the side of caution and flag all inconsistencies between the source code and protocol for further review since these can greatly increase the risk of actual vulnerabilities.

Pinocchio Protocol Inconsistencies. In the Pinocchio key generator, there exist values $\overrightarrow{A}$, $\overrightarrow{B}$, and $\overrightarrow{C}$ that need to be extended via $A_{m+1} = B_{m+2} = C_{m+3}$ and $A_{m+2} = A_{m+3} = B_{m+1} = B_{m+3} = C_{m+1} = C_{m+2}$ per the specification [84]. However, our Value Checker found that `libsnark` only extended $\overrightarrow{A}$, $\overrightarrow{B}$, and $\overrightarrow{C}$ via $A_{m+1} = B_{m+1} = C_{m+1}$ as an optimization for space complexity. After a review of the `libsnark` source code and protocol, we believe this inconsistency will not lead to a security vulnerability given how the values are used in practice. However, this instance is more subtle and less well-known than the previous one, so while it might not be exploitable, we believe inconsistencies of this nature are still valuable to flag for further review as this might not always be the case.

Toxic Waste. As part of the zkSNARK setup process, a set of private parameters informally referred to as "toxic waste" are generated. This toxic waste must be destroyed after the setup process, as possession of it allows one to forge proofs. However, our SnarkFuzzer identified an actual implementation error in a zkSNARK library called `playsnark`, whereby toxic waste is not properly destroyed during the setup phase. `playsnark` [124] serves as a learning playground for proof systems, including Pinocchio and Groth16. While `playsnark` makes no claims that it should be used for security-sensitive applications, we feel that this example still exemplifies valuable use cases for `SNARKProbe`. First, it can indeed catch errors that would

be exploitable in real applications. And second, while libraries like `Bellman` and `libsnark` are professionally audited, this will likely not always be the case for all zkSNARK code in production usage in the future (as we have seen in many other domains). Tooling such as `SNARKProbe` can be an invaluable resource for non-experts and small organizations, enabling them to assess the security posture of unaudited zkSNARK libraries, audit their own code throughout the development process, and facilitating informed decisions regarding future security implementations.

`libsnark` Out of Memory. SnarkFuzzer detects a corner-case software error in `libsnark`. When our Input Generator tried to produce a large R1CS matrix with tens of thousands of variables and constraints, we discovered that `libsnark` cannot compile and produce a zkSNARK program with such a large R1CS matrix without throwing a segmentation fault or memory error. Intuitively, this occurs because the input matrix we generated was quite dense, i.e., it had a large number of both variables and constraints, as opposed to being quite sparse like most typical R1CS matrices. `libsnark`'s R1CS storage format is clearly not setup to handle such cases. In general, this would not impact the typical usage of the `libsnark`, as our fuzzer is designed to look for extreme cases like this that might not happen with a typical real world proof statement.

We take this example as a chance to take a step back and revisit our comparison of SnarkFuzzer to a traditional, more generic fuzzer like AFL. As generic memory and software errors such as this have nothing to do with the actual cryptography and zkSNARK logic, traditional fuzzers are also able to detect them. However, it is likely to take a generic fuzzer significantly longer (and many more inputs) to catch such errors, as they lack the context to understand what an "extreme" input means in this instance, and, as we have seen, it is not easy to instrument a fuzzer with this context. Additionally, all previous instances that we discussed would not be detectable by a generic fuzzer, as they are specifically cryptographic logic errors.

3.6.3 Found in Previous Versions of Libraries

CVE-2019-7167 Problem. Finally, we discuss how `SNARKProbe` (specifically SnarkFuzzer) is able to detect real world vulnerabilities by demonstrating its ability to pinpoint a previous CVE. CVE-2019-7167 [125] was found in `libsnark` through a manual code audit, as part of its usage in the popular privacy-preserving cryptocurrency Zcash [70] (and has since been patched). At a high level, this vulnerability produced a bypass element in the key generation that damaged the soundness of the zkSNARK proof system. To fix this vulnerability, the value of $pk_A^{'}$ must be replaced from $\{A_{\mathrm{i}}(\tau)\alpha_A\rho_A\mathcal{P}\}_{\mathrm{i}=0}^{m+3}$ to $\{A_{\mathrm{i}}(\tau)\alpha_A\rho_A\mathcal{P}\}_{\mathrm{i}=n+1}^{m+3}$, with all other values the same.

To test this, we downloaded a version of `libsnark` from 2018 (Commit hash: bf2146b). When running SnarkFuzzer, the Value Checker *automatically* found this inconsistency and vulnerability in key generation. Additionally, while `libsnark` has since fixed this issue, it did not directly modify the structure of $pk_A^{'}$ in order to keep consistency with the structure of the other proving keys. Therefore, $pk_A^{'}$ still has size $m+3$ instead of $(m+3)-(n+1)$. `libsnark` just added an extra handler to reformat the $pk_A^{'}$ size in the prover, and while this does not appear to have caused any additional issues, it is unclear if this could result in other corner case issues.

3.7 Conclusion

In this paper we present `SNARKProbe`, a framework that can automatically and systematically check for potential software errors and cryptographic logic errors, as well as for consistency in the implemented proof statement, in R1CS-based zkSNARK libraries, with little manual input from the user. We demonstrated `SNARKProbe`'s design flexibility by implementing it for multiple different zkSNARK protocols and libraries. In addition, we performed extensive performance evaluations, as well as tested its ability to detect potential errors and inconsistencies. We believe `SNARKProbe` is a step in the right direction for ensuring the correctness of existing zkSNARK implementations and applications.

4. GREY-BOX DIFFERENTIAL FUZZING FOR ZERO-KNOWLEDGE PROOF BINARY APPLICATIONS

This chapter is based on joint work with Christina Garman (Purdue University)

5. ETHICS

6. CONCLUSION & FUTURE WORK

REFERENCES

[1] Y. Fan, P. Biswas, and C. Garman, "R+ r: A systematic study of cryptographic function identification approaches in binaries," in *2024 Annual Computer Security Applications Conference (ACSAC)*, IEEE, 2024, pp. 1092–1108.

[2] Y. Fan, P. Biswas, and C. Garman, "Evaluating approaches for identifying cryptographic functions in binaries,"

[3] *Openssl*, https://www.openssl.org/.

[4] *Wannacry ransomware*, https://en.wikipedia.org/wiki/WannaCry_ransomware_attack.

[5] *Petya ransomware*, https://www.proofpoint.com/us/glossary/petya.

[6] W. Yan, Z. Zhang, and N. Ansari, "Revealing packed malware," *IEEE S&P*, 2008.

[7] I. Harvey, "Cipher hunting: How to find cryptographic algorithms in large binaries," *NCipher Corporation Ltd*, 2001.

[8] Z. Lin, X. Jiang, D. Xu, and X. Zhang, "Automatic protocol format reverse engineering through context-aware monitored execution.," in *NDSS*, 2008.

[9] J. Caballero, H. Yin, Z. Liang, and D. Song, "Polyglot: Automatic extraction of protocol message format using dynamic binary analysis," in *ACM CCS*, 2007.

[10] Z. Wang, X. Jiang, W. Cui, X. Wang, and M. Grace, "Reformat: Automatic reverse engineering of encrypted messages," in *ESORICS*, 2009.

[11] R. Chang, L. Jiang, H. Shu, and H. He, "Cryptographic algorithms analysis technology research based on functions signature recognition," in *CIS*, 2014.

[12] P. Lestringant, F. Guihéry, and P.-A. Fouque, "Automated identification of cryptographic primitives in binary code with data flow graph isomorphism," in *ASIACCS*, 2015.

[13] *Draft crypto analyzer*, http://www.literatecode.com/draca.

[14] *Kanal - krypto analyzer for peid*, http://www.dcs.fmph.uniba.sk/zri/6.prednaska/tools/PEiD/plugins/kanal.htm.

[15] *Kerckhoffs*, https://github.com/felixgr/kerckhoffs.

[16] *Hash & crypto detector (hcd)*, https://webscene.ir/tools/show/Hash-and-Crypto-Detector-1.4.

[17] *Signsrch*, http://aluigi.altervista.org/mytoolz/signsrch.zip.

[18] *Findcrypt2*, http://www.hexblog.com/?p=28.

[19] J. Calvet, J. M. Fernandez, and J.-Y. Marion, "Aligot: Cryptographic function identification in obfuscated binary programs," in *ACM CCS*, 2012.

[20] D. Xu, J. Ming, and D. Wu, "Cryptographic function detection in obfuscated binaries via bit-precise symbolic loop mapping," in *IEEE S&P*, 2017.

[21] X. Li, X. Wang, and W. Chang, "Cipherxray: Exposing cryptographic operations and transient secrets from monitored binary execution," *IEEE TDSC*, 2012.

[22] N. Lutz, "Towards revealing attacker's intent by automatically decrypting network traffic," *Mémoire de maıtrise, ETH Zürich, Switzerland*, 2008.

[23] F. Gröbert, C. Willems, and T. Holz, "Automated identification of cryptographic primitives in binary programs," in *RAID*, 2011.

[24] J. Park and Y. Park, "Symmetric-key cryptographic routine detection in anti-reverse engineered binaries using hardware tracing," *Electronics*, 2020.

[25] E. C. R. Shin, D. Song, and R. Moazzezi, "Recognizing functions in binaries with neural networks," in *USENIX Security*, 2015.

[26] J. L. Wright and M. Manic, "Neural network approach to locating cryptography in object code," in *ETFA*, 2009.

[27] L. Benedetti, A. Thierry, and J. Francq, "Detection of cryptographic algorithms with grap.," *IACR Cryptology ePrint Archive*, 2017.

[28] A. Aigner, "Falke-mc: A neural network based approach to locate cryptographic functions in machine code," in *ARES*, 2018.

[29] G. Hill and X. Bellekens, "Cryptoknight: Generating and modelling compiled cryptographic primitives," *Information*, 2018.

[30] L. Jia, A. Zhou, P. Jia, L. Liu, Y. Wang, and L. Liu, "A neural network-based approach for cryptographic function detection in malware," *IEEE Access*, 2020.

[31] S. Lee, N.-s. Jho, D. Chung, Y. Kang, and M. Kim, "Rcryptect: Real-time detection of cryptographic function in the user-space filesystem," *Computers & Security*, 2022.

[32] *Data flow graph*, http://bears.ece.ucsb.edu/research-info/DP/dfg.html.

[33] *S-box*, https://en.wikipedia.org/wiki/S-box.

[34] *P-box*, https://en.wikipedia.org/wiki/Permutation_box.

[35] *Control-flow flattening*, https://github.com/obfuscator-llvm/obfuscator/wiki/Control-Flow-Flattening.

[36] C. Meijer, V. Moonsamy, and J. Wetzels, "Where's crypto?: Automated identification and classification of proprietary cryptographic primitives in binary code," in *USENIX Security*, 2021.

[37] P. Shirani, L. Wang, and M. Debbabi, "Binshape: Scalable and robust binary library function identification using function shape," in *DIMVA*, 2017.

[38] F. Matenaar, A. Wichmann, F. Leder, and E. Gerhards-Padilla, "Cis: The crypto intelligence system for automatic detection and localization of cryptographic functions in current malware," in *MALWARE*, 2012.

[39] *Cryptographic standards and guidelines*, https://csrc.nist.gov/projects/cryptographic-standards-and-guidelines.

[40] Association for Computing Machinery, *Artifact Review and Badging Policy*, Accessed: 2024-09-16, 2024. [Online]. Available: https://www.acm.org/publications/policies/artifact-review-and-badging-current.

[41] D. Andriesse, A. Slowinska, and H. Bos, "Compiler-agnostic function detection in binaries," in *EuroS&P*, 2017.

[42] X. Ren, M. Ho, J. Ming, Y. Lei, and L. Li, "Unleashing the hidden power of compiler optimization on binary code difference: An empirical study," in *PLDI*, 2021.

[43] A. Hansberry, A. Lasse, and A. Tarrh, "Cryptolocker: 2013's most malicious malware," *Retrieved February*, 2014.

[44] K. Cabaj, P. Gawkowski, K. Grochowski, and D. Osojca, "Network activity analysis of cryptowall ransomware," *Przeglad Elektrotechniczny*, 2015.

[45] S. Gallagher, *"locky" crypto-ransomware rides in on malicious word document macro*, Feb. 2016. [Online]. Available: https://arstechnica.com/security/2016/02/locky-crypto-ransomware-rides-in-on-malicious-word-document-macro/.

[46] Y. Lemmou and E. M. Souidi, "Infection, self-reproduction and overinfection in ransomware: The case of teslacrypt," in *Cyber Security*, 2018.

[47] M. Corporation, *Microsoft*. [Online]. Available: https://www.microsoft.com/en-us/wdsi/threats/malware-encyclopedia-description?Name=Win32%2FDircrypt.

[48] R. Harkanson and Y. Kim, "Applications of elliptic curve cryptography: A light introduction to elliptic curves and a survey of their applications," in *CISRC*, 2017.

[49] M. Al-Zubaidie, Z. Zhang, and J. Zhang, "Efficient and secure ecdsa algorithm and its applications: A survey," *arXiv preprint arXiv:1902.10313*, 2019.

[50] C. Collberg, C. Thomborson, and D. Low, "A taxonomy of obfuscating transformations," Department of Computer Science, The University of Auckland, New Zealand, Tech. Rep., 1997.

[51] *Control flow graph*, https://en.wikipedia.org/wiki/Control-flow_graph.

[52] Y. Li, Z. Sha, X. Xiong, and Y. Zhao, "Code obfuscation based on inline split of control flow graph," in *ICAICA*, 2021.

[53] D. Xu, "Opaque predicate: Attack and defense in obfuscated binary code," 2018.

[54] H. Xu, Y. Zhou, J. Ming, and M. Lyu, "Layered obfuscation: A taxonomy of software obfuscation techniques for layered security," *Cybersecurity*, 2020.

[55] Y. David, N. Partush, and E. Yahav, "Similarity of binaries through re-optimization," in *PLDI*, 2017.

[56] A. Saebjornsen, *Detecting fine-grained similarity in binaries*. University of California, Davis, 2014.

[57] H. Liu, Z. Yang, C. Liu, Y. Jiang, W. Zhao, and J. Sun, "Eclone: Detect semantic clones in ethereum via symbolic transaction sketch," in *ESEC/FSE*, 2018.

[58] M. Chandramohan, Y. Xue, Z. Xu, Y. Liu, C. Y. Cho, and H. B. K. Tan, "Bingo: Cross-architecture cross-os binary search," in *FSE*, 2016.

[59] D. McKee, N. Burow, and M. Payer, "Software ethology: An accurate, resilient, and cross-architecture binary analysis framework," *arXiv preprint arXiv:1906.02928*, 2019.

[60] S. H. Ding, B. C. Fung, and P. Charland, "Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization," in *IEEE S&P*, 2019.

[61] J. Ming, D. Xu, Y. Jiang, and D. Wu, "{Binsim}: Trace-based semantic binary diffing via system call sliced segment equivalence checking," in *USENIX Security*, 2017.

[62] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, "Neural network-based graph embedding for cross-platform binary code similarity detection," in *ACM CCS*, 2017.

[63] Y. Fan, Y. Xu, and C. Garman, "Snarkprobe: An automated security analysis framework for zksnark implementations," in *International Conference on Applied Cryptography and Network Security*, Springer, 2024, pp. 340–372.

[64] J. Nicas, M. Isaac, and S. Frenkel, *Millions flock to telegram and signal as fears grow over big tech*, https://www.nytimes.com/2021/01/13/technology/telegram-signal-apps-big-tech.html, 2021.

[65] A. Macgillivray and T. deBlanc-Knowles, *Advancing a vision for privacy enhancing technologies*, https://www.whitehouse.gov/ostp/news-updates/2022/06/28/advancing-a-vision-for-privacy-enhancing-technologies/, 2022.

[66] O. Goldreich, S. Micali, and A. Wigderson, "Proofs that yield nothing but their validity and a methodology of cryptographic protocol design," in *FOCS*, 1986.

[67] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza, *Snarks for c: Verifying program executions succinctly and in zero knowledge*, Cryptology ePrint Archive, Report 2013/507, 2013.

[68] B. Parno, J. Howell, C. Gentry, and M. Raykova, "Pinocchio: Nearly practical verifiable computation," in *IEEE S&P*, 2013.

[69] J. Groth, "On the size of pairing-based non-interactive arguments," in *Eurocrypt*, 2016.

[70] *Zcash*, https://z.cash/, 2023.

[71] *Dark forest*, https://blog.zkga.me/, 2022.

[72] *Aleo*, https://www.aleo.org/, 2023.

[73] *Zksnarks for the world*, https://research.protocol.ai/sites/snarks/, 2023.

[74] *Zero-knowledge rollups*, https://ethereum.org/en/developers/docs/scaling/zk-rollups/, 2022.

[75] C. Garman, M. Green, G. Kaptchuk, I. Miers, and M. Rushanan, "Dancing on the lip of the volcano: Chosen ciphertext attacks on apple {imessage}," in *USENIX Security*, 2016.

[76] M. Naveed, S. Kamara, and C. V. Wright, "Inference attacks on property-preserving encrypted databases," in *ACM CCS*, 2015.

[77] NSA, *Patch critical cryptographic vulnerability in microsoft windows clients and servers*, https://media.defense.gov/2020/Jan/14/2002234275/-1/-1/0/CSA-WINDOWS-10-CRYPT-LIB-20190114.PDF, 2020.

[78] D. Rupprecht, K. Kohls, T. Holz, and C. Pöpper, "Call me maybe: Eavesdropping encrypted {lte} calls with {revolte}," in *USENIX Security*, 2020.

[79] *Zcash counterfeiting vulnerability successfully remediated*, https://electriccoin.co/blog/zcash-counterfeiting-vulnerability-successfully-remediated/, 2019.

[80] *Fixing vulnerabilities in the zcash protocol*, https://electriccoin.co/blog/fixing-zcash-vulns/, 2016.

[81] *Nu4 cryptographic specification and implementation review*, https://research.nccgroup.com/wp-content/uploads/2020/09/NCC_Group_Zcash_ZCHX006_Report_2020-09-03_v2.0.pdf, 2020.

[82] J.-P. Aumasson and Y. Romailler, "Automated testing of crypto software using differential fuzzing," *Black Hat USA*, 2017.

[83] *Project everest*, https://project-everest.github.io/, 2023.

[84] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, "Succinct non-interactive zero knowledge for a von neumann architecture," in *USENIX Security*, 2014.

[85] *libsnark: A c++ library for zksnark proofs*, https://github.com/scipr-lab/libsnark, 2020.

[86] *bellman, a zk-snark library*, https://github.com/zkcrypto/bellman, 2023.

[87] *libsnark: A rust implementation of the groth16 zksnark*, https://github.com/arkworks-rs/groth16, 2023.

[88] *gnark zk-snark library*, https://github.com/Consensys/gnark, 2023.

[89] C. Chen, B. Cui, J. Ma, R. Wu, J. Guo, and W. Liu, "A systematic review of fuzzing techniques," *Computers & Security*, 2018.

[90] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as markov chain," *IEEE TSE*, 2017.

[91] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *ACM CCS*, 2017.

[92] A. D. Householder and J. M. Foote, "Probability-based parameter selection for black-box fuzz testing," Carnegie Mellon University, SEI, Tech. Rep., 2012.

[93] M. Woo, S. K. Cha, S. Gottlieb, and D. Brumley, "Scheduling black-box mutational fuzzing," in *ACM CCS*, 2013.

[94] P. Godefroid, H. Peleg, and R. Singh, "Learn&fuzz: Machine learning for input fuzzing," in *ASE*, 2017.

[95] M. Zalewski, *American fuzzy lop*, Available at https://github.com/mirrorer/afl. v2.52b Accessed July 2022, 2016.

[96] Google, *Syzkaller - kernel fuzzer*, Available at https://github.com/google/syzkaller. Accessed July 2022, 2017.

[97] R. Stevens, O. Suciu, A. Ruef, S. Hong, M. Hicks, and T. Dumitraş, "Summoning demons: The pursuit of exploitable bugs in machine learning," *arXiv preprint arXiv:1701.04739*, 2017.

[98] L. d. Moura and N. Bjørner, "Z3: An efficient smt solver," in *TACAS*, 2008.

[99] H. Barbosa, A. Reynolds, D. E. Ouraoui, C. Tinelli, and C. Barrett, "Extending smt solvers to higher-order logic," in *CADE*, 2019.

[100] L. De Moura and N. Bjørner, "Satisfiability modulo theories: Introduction and applications," *CACM*, 2011.

[101] M. Fredrikson and B. Livshits, "Zø: An optimizing distributing zero-knowledge compiler," in *USENIX Security*, 2014.

[102] o1-labs, *Snarky*, Available at https://github.com/o1-labs/snarky. Accessed August 2022, 2023.

[103] A. Kosba, *Xjsnark*, Available at https://github.com/akosba/xjsnark. Accessed August 2022, 2022.

[104] iden3, *Circom - curcuit compiler for zk proving systems*, Available at https://github.com/iden3/circom. Accessed August 2022, 2023.

[105] J. Somorovsky, "Systematic fuzzing and testing of tls libraries," in *ACM CCS*, 2016.

[106] A. Walz and A. Sikora, "Exploiting dissent: Towards fuzzing-based differential black-box testing of tls implementations," *IEEE TDSC*, 2017.

[107] N. Mouha, M. S. Raunak, D. R. Kuhn, and R. Kacker, "Finding bugs in cryptographic hash function implementations," *IEEE transactions on reliability*, 2018.

[108] *Clang's source-based code coverage*, https://clang.llvm.org/docs/SourceBasedCodeCoverage.html, 2023.

[109] Q.-S. Phan, L. Bang, C. S. Pasareanu, P. Malacaria, and T. Bultan, "Synthesis of adaptive side-channel attacks," in *IEEE CSF*, 2017.

[110] C. S. Pasareanu, Q.-S. Phan, and P. Malacaria, "Multi-run side-channel analysis using symbolic execution and max-smt," in *IEEE CSF*, 2016.

[111] L. Bang, N. Rosner, and T. Bultan, "Online synthesis of adaptive side-channel attacks based on noisy observations," in *IEEE EuroS&P*, 2018.

[112] G. Beck, M. Zinkus, and M. Green, "Automating the development of chosen ciphertext attacks," in *USENIX Security*, 2020.

[113] D. Kupser, C. Mainka, J. Schwenk, and J. Somorovsky, "How to break {xml} encryption–automatically," in *USENIX WOOT*, 2015.

[114] M. Barbosa *et al.*, "Sok: Computer-aided cryptography," in *IEEE S&P*, 2021.

[115] J. Protzenko, B. Beurdouche, D. Merigoux, and K. Bhargavan, "Formally verified cryptographic web applications in webassembly," in *IEEE S&P*, 2019.

[116] *atheris, atheris: A coverage-guided, native python fuzzer*, https://github.com/google/atheris, 2023.

[117] *doxygen, doxygen*, https://github.com/doxygen/doxygen, 2023.

[118] A. Nitulescu, "Zk-snarks: A gentle introduction," Technical report, Tech. Rep., 2020.

[119] Ethereum, *Python implementation of ecc pairing and bn_128 and bls12_381 curve operations*, Available at https://github.com/ethereum/py_ecc. Accessed Dec 2021, Ethereum, Dec. 2021.

[120] A. Faz-Hernández and K. Kwiatkowski, *Introducing circl: An advanced cryptographic library*, Available at https://github.com/cloudflare/circl. v1.2.0 Accessed Jun 2022, Cloudflare, Jun. 2019.

[121] *Snarkjs*, https://github.com/iden3/snarkjs, 2023.

[122] *Wasmsnark*, https://github.com/iden3/wasmsnark, 2023.

[123] *Rapidsnark*, https://github.com/iden3/rapidsnark, 2023.

[124] *Playsnark: A playground to learn proofs systems*, https://github.com/nikkolasg/playsnark, 2020.

[125] A. Gabizon, *Auroralight: Improved prover efficiency and srs size in a sonic-like system*, Cryptology ePrint Archive, Paper 2019/601, 2019. [Online]. Available: https://eprint.iacr.org/2019/601.

[126] *Ida pro*, https://hex-rays.com/ida-pro/.

A. Appendix for Chapter 2

A.1 List of Optimizations, Obfuscations, and their Combinations for Our Evaluation Metrics

0. No identification
1. Identification with optimization flag -O0
2. Identification with optimization flag -O1
3. Identification with optimization flag -O2
4. Identification with optimization flag -O3
5. Identification with optimization flag Os
6. Identification with optimization flag: -Ofast
7. Identification with instruction substitution obfuscation: -obs_sub
8. Identification with control-flow flattening: -obs_fla
9. Identification with bogus control-flow: -obs_bcf
10. Identification with combination of obfuscation: -obs_sub, obs_fla, obs_bcf
11. Identification with combination of obfuscation and optimization: -obs_sub, obs_fla, obs_bcf, -O3

A.2 Procedure for R+R Evaluation

A.2.1 Steps in our Reproduction Evaluation

1. Obtain access to each tool; compile the tool if required, following the provided instructions.
2. Set up the environment to replicate the tool's original configuration (follows the same operating conditions).

3. Obtain the original test cases for each tool. If unavailable, the reproduction evaluation will be skipped (follows the same measuring system).

4. Execute each tool with the provided test cases to reproduce the results (follows the same measurement procedure).

A.2.2 Steps in our Replication Evaluation

1. Obtain access to each tool; recompile the tool on our system if the source code is available.

2. Build and run each tool on Windows 10, Ubuntu 18.04, and macOS Sonoma (follows the different measuring system).

3. Compile our custom benchmarks with the evaluation metrics into binaries for testing.

4. Execute each tool with our benchmarks to assess replication capability (follows the different location on multiple trials).

A.3 Tools

We provide introductions to the tools analyzed in this work. For each tool, we give a high level overview, highlight key features for a user, and provide a discussion on its strengths and weaknesses, as relevant.

A.3.1 Aligot

Summary. Aligot [19] is based on the assumption that the input-output relations in a cryptographic function remain unchanged even if there is obfuscation. It performs an inter-loop data-flow analysis to identify cryptographic parameters and then performs a comparison with a reference implementation. The authors successfully evaluated the tool to identify four different cryptographic functions: TEA, RC4, AES and MD5.

Highlights for User.

- Requires the user to compile the tool with Intel Pin

- Requires an earlier version of Intel Pin and Python library *networkx* (latest does not work)

Other Discussion.

- Aligot has not been updated to accommodate the latest dependency changes. Specifically, the Python library "networkx" uses different data types that cause crashes in Aligot
- Additionally, the Aligot tracer has not been adapted to the latest version of Intel Pin, and the tested version appears to be discontinued

A.3.2 CryptoHunt

Summary. CryptoHunt [20] introduces the *bit-precise symbolic loop mapping* technique to identify cryptographic functions. Using this technique the tool captures the semantics of possible cryptographic algorithms within a loop and then performs guided fuzzing to efficiently match boolean formulas with known reference implementations. It shows its efficacy on commonly used cryptographic functions such as TEA, AES, RC4, MD5, and RSA under different control and data obfuscation scheme combinations.

Highlights for User.

- Requires the user to compile the tool with Intel Pin (2.13, 3.2, and 3.28 are tested)
- Requires the user to indicate the reference loop
- Higher accuracy rate and detection results in obfuscated binaries

Other Discussion.

- Depending on the binary program size, CryptoHunt may find 10,000 loops in the binary, which needs to be compared with the reference one by one. The long analysis time may not suitable for a large program.

A.3.3 CryptoKnight

Summary. CryptoKnight [29] employs a Dynamic Convolutional Neural Network (DCNN) to learn from variable-length control flow diagnostics output obtained from a dynamic trace, and is able to detect AES, RC4, Blowfish, MD5, and RSA.

Highlights for User.

- Can easily integrate new samples through the scalable synthesis of customizable cryptographic algorithms
- Automated and limits user interaction
- High accuracy rate (96%)

Other Discussion.

- Has been evaluated on GnuPG and a real-world ransomware binary 'GonnaCry' and can successfully identify the use of AES and RSA

A.3.4 Draft Crypto Analyzer (DRACA)

Summary. DRACA [13] targets preliminary detection and analysis of cryptographic algorithms within executables. Although DRACA is implemented as a command line utility for x86/Win32, it can also analyze Unix ELF binaries, Java applets, as well as 16- and 32-bit DOS Windows executables.

Highlights for User.

- No compiling is needed
- Can be directly run by passing the target binary program

Other Discussion.

- DRACA can only provide a rough idea of what kind of algorithms to look at without actual spending time on decompilation and code analysis.
- Detection result cannot be used without further analysis.

A.3.5 FindCrypt2

Summary. FindCrypt2 [18] has been implemented as a plug-in support for IDA Pro [126] which searches constants used in the initialization of cryptographic algorithms. It can detect commonly used encryption algorithms.

Highlights for User.

- Requires IDA Pro, a commercial disassembler software, which is not free

A.3.6 HCD

Summary. HCD [16] detects roughly 90 hash and cryptographic algorithms and compiles for portable executable files with a GUI interface and Shell integration, Command line support.

Highlights for User.

- No compiling is needed
- Can be directly run by passing the target binary program

Other Discussion.

- Can only detect cryptographic functions in executable file for Microsoft Windows (.exe). This tool does not support the analysis of Unix based binary programs.

A.3.7 PEiD KANAL

Summary. PEiD KANAL [14] is a plug-in for PEiD, searching for cryptographic algorithms with a specific signature such as fixed s-boxes, permutation tables, and initialization values. It searches for connections to identified code or data and find its associated address.

Highlights for User.

- No compiling is needed
- Can be directly run by passing the target binary program

A.3.8 Signsrch

Summary. Signsrch [17] is another IDA Pro plug-in which aims to detect not just encryption algorithms, but also compression algorithms, multimedia, etc. The tool searches for known signatures as well as allows a user to add their own signature.

Highlights for User.

- No compiling is needed
- Can be directly run by passing the target binary program

A.3.9 Where's Crypto

Summary. Where's Crypto [36] is an IDA SDK based tool, aiming to automatically identify cryptographic primitives in binaries. Where's Crypto can detect cryptographic functions that has as-of-yet unknown primitives if it falls within a taxonomical class of well-defined primitives.

Highlights for User.

- Requires IDA Pro, a commercial disassembler software, which is not free
- Supports unknown cryptographic primitives

A.4 Additional Tables

This appendix contains the complete results of all of our replication and reproduction studies.

Tool		Cryptographic Algorithm									Micro-Benchmark				
		AES	DES	MD5	RC4	RC5	RSA	SHA1	SHA256	TEA	File	I/O	Math	Matrix	Network
DRACA	False Positive	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	False Negative	100%	100%	69%	0%	0%	0%	0%	0%	28%	0%	0%	0%	0%	0%
	True Positive	0%	0%	31%	0%	100%	0%	100%	0%	72%	0%	0%	0%	0%	0%
	True Negative	0%	0%	0%	100%	0%	100%	0%	100%	0%	100%	100%	100%	100%	100%
CryptoKnight	False Positive	0%	0%	0%	100%	0%	0%	0%	0%	0%	0%	0%	0%	74%	0%
	False Negative	0%	0%	100%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	True Positive	100%	0%	0%	0%	0%	100%	0%	0%	0%	0%	0%	0%	0%	0%
	True Negative	0%	100%	0%	0%	100%	0%	100%	100%	100%	100%	100%	100%	26%	100%
Findcrypt2	False Positive	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	False Negative	100%	0%	69%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	True Positive	0%	100%	31%	0%	100%	0%	100%	100%	0%	0%	0%	0%	0%	0%
	True Negative	0%	0%	0%	100%	0%	100%	0%	0%	100%	100%	100%	100%	100%	100%
Signsrch	False Positive	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	100%	0%
	False Negative	0%	100%	69%	0%	0%	0%	31%	0%	0%	0%	0%	0%	0%	0%
	True Positive	100%	0%	31%	0%	100%	0%	69%	100%	100%	0%	0%	0%	0%	0%
	True Negative	0%	0%	0%	100%	0%	100%	0%	0%	0%	100%	100%	100%	0%	100%
Where's Crypto	False Positive	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	False Negative	0%	0%	69%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
	True Positive	100%	100%	31%	0%	0%	0%	100%	100%	100%	0%	0%	0%	0%	0%
	True Negative	0%	0%	0%	100%	100%	100%	0%	0%	0%	100%	100%	100%	100%	100%

Table A.1. Performance Metrics for Tool Evaluation: False Positives, False Negatives, True Positives, and True Negatives

Tool	Aligot	Crypto-Hunt	Crypto-Knight	DRACA	FindCrypt2	FALKE-MC	HCD	Kerckhoff	PEiD KANAL	SignSrch	Softmax Classifier	Wheres Crypto
ADLER32							*Not Available*		*Not Available*		✓	
AES (Rijndael)	✓	✓	✓	✓	✓	✓		✓		✓		✓
BASE64										✓	✓	
Blowfish			✓	✓	✓					✓	✓	
Camellia					✓							
CAST					✓					✓		
CAST-256				✓	✓					✓		
CRC32				✓	✓					✓	✓	
DES				✓	✓			✓		✓	✓	✓
EC										✓		
GOST					✓					✓		
HAVAL					✓					✓	✓	
MARS				✓	✓					✓		
MD2					✓					✓		
MD4					✓					✓		
MD5	✓	✓	✓	✓	✓			✓		✓	✓	✓
RC2				✓	✓					✓		
RC4	✓	✓	✓			✓		✓				
RC5				✓	✓					✓	✓	
RC6				✓	✓					✓	✓	
Ripemd-160				✓						✓		
RSA	✓	✓	✓					✓				
SAFER				✓	✓					✓		
SHA-1				✓	✓					✓	✓	✓
SHA-256					✓					✓	✓	✓
SHA-512					✓					✓		✓
SHARK					✓					✓		
Skipjack				✓	✓					✓		
Square					✓					✓		
TEA	✓	✓		✓						✓		
Tiger				✓	✓					✓		
Twofish				✓	✓					✓		
WAKE					✓					✓		
Whirlpool					✓					✓		
XTEA												✓

Table A.2. Cryptographic algorithm detection support as stated in the paper or documentation for each tool

Table A.3. Detailed evaluation results for each tool based on our uniform benchmarking suite

Tool	Flag	Basic Cryptographic Function												Micro-Benchmark					Library											Large Project
		AES	DES	ECC	MD5	RC4	RC5	RSA	SHA1	SHA256	TEA	XTEA	XXTEA	File	I/O	Math	Matrix	Network	openssl	libgcrypt	libsodium	mbedTLS	gnuTLS	bzip2	zlib	ffmpeg	libgsm	libjpeg	libpng	signal
DRACA	-O0	○△□	○△□	○△□	●▲■	○△□	●▲■	○△□	●▲■	○△□	●▲■	○△□	●△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	●▲□
	-O1	○△□	○△□	○△□	○△■	○△□	●▲■	○△□	●▲■	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	●▲□
	-O2	○△□	○△□	○△□	○△■	○△□	●▲■	○△□	●▲■	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	●▲□
	-O3	○△□	○△□	○△□	○△■	○△□	●▲■	○△□	●▲■	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	●▲□
	-Os	○△□	○△□	○△□	○△■	○△□	●▲■	○△□	●▲■	○△□	●▲■	●▲□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	●▲□
	-Ofast	○△□	○△□	○△□	○△■	○△□	●▲■	○△□	●▲■	○△□	●▲■	●▲□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	●▲□
	obs_sub	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_fla	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_bcf	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_sub, obs_fla, obs_bcf	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_sub, obs_fla, obs_bcf, -O3	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
CryptoKnight	-O0	●▲□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□
	-O1	●▲□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□
	-O2	●▲□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□
	-O3	●▲□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□
	-Os	●▲□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□
	-Ofast	●▲□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	●▲□	○△□	○△□	○△□	○△□	○△□	○△□	●▲□	○△□	○△□	○△□	○△□
	obs_sub	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚
	obs_fla	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚
	obs_bcf	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚
	obs_sub, obs_fla, obs_bcf	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚
	obs_sub, obs_fla, obs_bcf, -O3	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚
Findcrypt2	-O0	○△□	●▲■	○△□	●▲■	○△□	●▲■	○△□	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-O1	○△□	●▲■	○△□	○△■	○△□	●▲■	○△□	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-O2	○△□	●▲■	○△□	○△■	○△□	●▲■	○△□	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-O3	○△□	●▲■	○△□	○△■	○△□	●▲■	○△□	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-Os	○△□	●▲■	○△□	○△■	○△□	●▲■	○△□	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-Ofast	○△□	●▲■	○△□	○△■	○△□	●▲■	○△□	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■
	obs_sub	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_fla	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_bcf	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_sub, obs_fla, obs_bcf	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_sub, obs_fla, obs_bcf, -O3	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
HCD	-O0	○△■	○△□	○△□	○△■	○△□	○△□	○△□	○△■	○△■	○△■	○△■	○△■	○△□	○△□	○△□	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△□	○△■
	-O1	○△■	○△□	○△□	○△■	○△□	○△□	○△□	○△■	○△■	○△■	○△■	○△■	○△□	○△□	○△□	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△□	○△■
	-O2	○△■	○△□	○△□	○△■	○△□	○△□	○△□	○△■	○△■	○△■	○△■	○△■	○△□	○△□	○△□	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△□	○△■
	-O3	○△■	○△□	○△□	○△■	○△□	○△□	○△□	○△■	○△■	○△■	○△■	○△■	○△□	○△□	○△□	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△□	○△■
	-Os	○△■	○△□	○△□	○△■	○△□	○△□	○△□	○△■	○△■	○△■	○△■	○△■	○△□	○△□	○△□	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△□	○△■
	-Ofast	○△■	○△□	○△□	○△■	○△□	○△□	○△□	○△■	○△■	○△■	○△■	○△■	○△□	○△□	○△□	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△■	○△■	○△■	○△□	○△□	○△□	○△■
	obs_sub	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚
	obs_fla	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚
	obs_bcf	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚
	obs_sub, obs_fla, obs_bcf	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚
	obs_sub, obs_fla, obs_bcf, -O3	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚
PEiD KANAL	-O0	●▲■	●▲■	○△□	●▲■	○△□	●▲■	○△□	●▲■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-O1	●▲■	●▲■	○△□	○△■	○△□	●▲■	○△□	●▲■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-O2	●▲■	●▲■	○△□	○△■	○△□	●▲■	○△□	●▲■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-O3	●▲■	●▲■	○△□	○△■	○△□	●▲■	○△□	●▲■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-Os	●▲■	●▲■	○△□	○△■	○△□	●▲■	○△□	●▲■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-Ofast	●▲■	●▲■	○△□	○△■	○△□	●▲■	○△□	●▲■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	obs_sub	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_fla	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_bcf	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_sub, obs_fla, obs_bcf	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_sub, obs_fla, obs_bcf, -O3	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚
Signsrch	-O0	●▲■	○△□	○△□	●▲■	○△□	●▲■	○△□	●▲■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	○△□	○△□	●▲■	●▲■	○△□	○△□	●▲□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-O1	●▲■	○△□	○△□	○△■	○△□	●▲■	○△□	●▲■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	○△□	○△□	●▲■	●▲■	○△□	○△□	●▲□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-O2	●▲■	○△□	○△□	○△■	○△□	●▲■	○△□	●△■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	○△□	○△□	●▲■	●▲■	○△□	○△□	●▲□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-O3	●▲■	○△□	○△□	○△■	○△□	●▲■	○△□	○△■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	○△□	○△□	●▲■	●▲■	○△□	○△□	●▲□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-Os	●▲■	○△□	○△□	○△■	○△□	●▲■	○△□	●△■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	○△□	○△□	●▲■	●▲■	○△□	○△□	●△□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	-Ofast	●▲■	○△□	○△□	○△■	○△□	●▲■	○△□	○△■	●▲■	●▲■	●▲■	●▲■	○△□	○△□	○△□	●▲■	○△□	○△□	●▲■	●▲■	○△□	○△□	●▲□	●▲■	●▲■	○△□	○△□	○△□	●▲■
	obs_sub	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_fla	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_bcf	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_sub, obs_fla, obs_bcf	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
	obs_sub, obs_fla, obs_bcf, -O3	○▲⬚	○△⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚	○▲⬚	○▲⬚	○△⬚	○△⬚	○△⬚	○▲⬚
Where's Crypto	-O0	●▲■	●▲■	○△□	●▲■	○△□	○△□	○△□	●▲■	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	●▲■
	-O1	●▲■	●▲■	○△□	○△■	○△□	○△□	○△□	●▲■	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	●▲■
	-O2	●▲■	●▲■	○△□	○△■	○△□	○△□	○△□	●▲■	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	●▲■
	-O3	●▲■	●▲■	○△□	○△■	○△□	○△□	○△□	●▲■	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	●▲■
	-Os	●▲■	●▲■	○△□	○△■	○△□	○△□	○△□	●▲■	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	●▲■	○△□	○△□	○△□	●▲■
	-Ofast	●▲■	●▲■	○△□	○△■	○△□	○△□	○△□	●▲■	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	○△□	●▲■	●▲■	●▲■	○△□	○△□	○△□	○△□	●

B. Appendix for Chapter 3

B.1 Additional Information for Constraint Checker

B.1.1 An optimization for the SMT solver

The SMT solver may take a while to solve some complicated equations. To reduce the processing time in the equality check, we introduce optimizations and simplify some equations in the R1CS matrix. For example, many `libsnark` gadgets use bit shifting, which produces the equation $(x * (1 + (p - 1) * x)) \bmod p = 0$ in R1CS gates. This equation represents $x = 0$ or $x = 1$. For our optimization, if the Constraint Checker detects such a relation, it will be replaced with $0 \leq x \wedge x \leq 1$ where $x \in \mathbb{F}$. These are logically equivalent, but our replacement is much easier for the SMT solver to work with. A real world gadget like the `comparison_gadget` in `libsnark` has 16 constraints, and there are 11 constraints representing $x = 0$ or $x = 1$ (see Appendix B.3).

B.1.2 Protocol of the Constraint Checker by the SMT Solver

(a) Parameters.

A set of statement equations E_1 in size p where equation $\mathrm{e}_1^{\mathrm{i}} \in E_1$ and $\mathrm{i} \in [1, p]$.

A set of equations E_2 converted from R1CS gates in size q where $\mathrm{e}_2^{\mathrm{i}} \in E_2$ and $\mathrm{i} \in [1, q]$.

m private variables x_1^{i} in E_1 where $\mathrm{i} \in [1, m]$ and n private variables x_2^{i} in E_2 where $\mathrm{i} \in [1, n]$ and $n \geq m$. A set of auxiliary input v_x^{i}.

k public variables y_1^{i} in E_1 and k public variables y_2^{i} in E_2 where $\mathrm{i} \in [1, k]$. A set of primary input v_y^{i}.

A SMT Pro Solver S and Optimizer O with modulus P. $0 \leq x_1 < P$, $0 \leq x_2 < P$, $0 \leq y_1 < P$, and $0 \leq y_2 < P$ without optimization.

(b) Function valid check.

INPUTS: E_1, E_2, S.

OUTPUTS: Boolean result of function validation.

1. Check and solve the R1CS gates:

   Set a solvers $S_1 \in S$.

   Add $\mathrm{e}_1^{\mathrm{i}} \in E_1$ where $\mathrm{i} \in [1, p]$ and range of x_1^{i}, x_2^{i} to S_1.

   Solve S_1 and get the set of private variables solution as $M_{1_x}^{\mathrm{i}}$ and set of public variables solution as $M_{1_y}^{\mathrm{i}}$.

2. Check and solve the statement equations:

   Set a solvers $S_2 \in S$.

   Add $\mathrm{e}_2^{\mathrm{i}} \in E_1$ where $\mathrm{i} \in [1, q]$ and range of y_1^{i}, y_2^{i} to S_2.

   Solve S_2 and get the set of private variables solution as $M_{2_x}^{\mathrm{i}}$ and set of public variables solution as $M_{2_y}^{\mathrm{i}}$.

3. Output $v_x^{\mathrm{i}} \in M_{1_x}^{\mathrm{i}}$ *AND* $v_x^{\mathrm{i}} \in M_{2_x}^{\mathrm{i}}$ *AND* $v_y^{\mathrm{i}} \in M_{1_y}^{\mathrm{i}}$ *AND* $v_y^{\mathrm{i}} \in M_{2_y}^{\mathrm{i}}$.

(c) Equivalent comparison.

INPUTS: E_1, E_2, S.

OUTPUTS: Boolean result of function equality.

Set two solvers S_1 and $S_2 \in S$.

1. Comparing with public variables:

   Add $\mathrm{e}_1^{\mathrm{i}} \in E_1$ where $\mathrm{i} \in [1, p]$ and range of x_1^{i}, y_1^{i} to S_1.

   Add $\mathrm{e}_2^{\mathrm{i}} \in E_2$ where $\mathrm{i} \in [1, q]$ and range of x_2^{i}, y_2^{i} to S_1.

   Add $x_1^{\mathrm{i}} == x_2^{\mathrm{i}}$ where $\mathrm{i} \in [1, m]$ to S_1.

   Add $y_1^{\mathrm{i}} \neq y_2^{\mathrm{i}}$ where $\mathrm{i} \in [1, k]$ to S_1.

   Check if S_1 is *SAT*, *UNSAT*, or *UNSURE* as c_1.

2. Comparing with Boolean variables:

   Set b_1 and $b_2 \in$ BOOLEAN

   Add $b_1 = AND(E_1)$ to S_2.

   Add $b_2 = AND(E_2)$ to S_2.

   Add $x_1^{\mathrm{i}} == x_2^{\mathrm{i}}$ where $\mathrm{i} \in [1, m]$ to S_2.

   Add $y_1^{\mathrm{i}} == y_2^{\mathrm{i}}$ where $\mathrm{i} \in [1, k]$ to S_2.

   Add $b_1 \neq b_2$ to S_2.

   Check if S_2 is *SAT*, *UNSAT*, or *UNSURE* as c_2.

3. Output ($c_1 == UNSAT\ AND\ c_2 == UNSAT$).

(d) Domain comparison.

INPUTS: E_1, E_2, S, O.

OUTPUTS: Boolean result of domain comparison.

1. Comparing upper and lower bond:

   Add $\mathrm{e}_1^{\mathrm{i}} \in E_1$ where $\mathrm{i} \in [1, p]$ and range of x_1^{i}, x_2^{i} to O.

   Add $\mathrm{e}_2^{\mathrm{i}} \in E_1$ where $\mathrm{i} \in [1, q]$ and range of y_1^{i}, y_2^{i} to O.

   Solve the upper bond u_1 and lower bond l_1 from O

   Solve the upper bond u_2 and lower bond l_2 from O

2. Check if R1CS is valid outside statement domain:

   Add *NOT*(*AND*(range of x_1^{i}, x_2^{i})) to S.

   Add $\mathrm{e}_2^{\mathrm{i}} \in E_1$ where $\mathrm{i} \in [1, q]$ and range of x_2^{i}, y_2^{i} to S.

   Check if S is *SAT*, *UNSAT*, or *UNSURE* as c.

3. Output ($c == UNSAT\ AND\ u_1 == u_2\ AND\ l_1 == l_2$).

B.1.3 Example Constraint Checker

For the interested reader, we provide an example of our Constraint Checker to show how a developer can run the tool. More examples can be found in our code.

```
x = z3.Int("x")
## Indicate the public variables and private variables
allocate = ["x"]
variables = [x]
## Indicate the statement of a proof
statement = [x < 60]
## Provide the snark program to automatically extract the R1CS matrix
## Developer can also manually provide the R1CS matrix
path = os.path.join(currentdir, "range")
## Call the functions in ConstraintChecker to evaluate the correctness
fcmp = FunctionComparison(path)
fcmp.allocate(allocate)
fcmp.set_input_sizes(0)
fcmp.addVariables(variables)
fcmp.addStatement(statement)
fcmp.addGadget(gadget1.comparison_gadget("max")) ## Optional
fcmp.addRange(x, z3.And(x < 60))
fcmp.runComparisonTests()
```

B.1.4 Domain Gap Issues

The same upper bound and lower bound does not guarantee that statement equations and R1CS gates have the same range. Figure B.1c and B.1d shows an example of an unmatched domain with the same upper bound and lower bound.

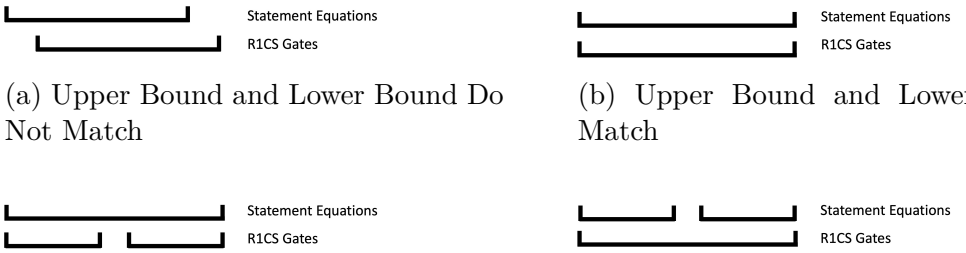
Domain Gap in Statement Equations

The ground truth domain of statement equations is known and provided by the user (which means we know the domain gap in the statement equations), the Constraint Checker can use the SMT solver to find if the R1CS gates have solution(s) outside the domain of statement equations (e.g. in the gap). If the SMT solver returns `SAT`, then the domains' of the statement equations and R1CS gates do not match and thus they are not equivalent. This example corresponds to Figure B.1d.

Domain Gap in R1CS Gates

Since the domain of the R1CS gates is unknown, the Constraint Checker cannot use the SMT solver to detect an unmatched domain as in the previous example. However, in this instance, the prover likely will not be able to generate a proof for a secret value in the gap because these secret values are not in the domain of R1CS Gates even though they are valid solutions to the original statement equations. That is, the set of allowable input values for the implemented R1CS matrix is a subset of the set of input values for the desired proof statement. Therefore, while this leads to undesirable behavior as the prover cannot generate proofs for the entire valid input range, it does not lead to any exploits or "fake" proofs (i.e., the ability to generate a valid proof without knowledge of the secret value). This example corresponds to Figure B.1c.

We believe this type of gap is very rare, but we still try to find this issue by producing a set of uniform random numbers as input for statement equations and R1CS gates. If the R1CS gates do not have a solution but the statement equations do have a valid solution for an input number, then there is a gap in the statement equations, which is not equivalent to the statement equations' domain. This test does not guarantee finding an existing domain gap, but a larger set of numbers has higher confidence in finding any existing gaps.



(a) Upper Bound and Lower Bound Do Not Match

(b) Upper Bound and Lower Bound Match

(c) R1CS Gates Has Gap Between Bound

(d) Statement Equations Has Gap Between Bound

Figure B.1. Examples of Domain Comparison

B.2 Additional Information for SnarkFuzzer

B.2.1 Example Ideal Model Files

```
Relative Path: knowledge_commitment/kc_multiexp.tcc
FUNCTION,2,99,127,kc_batch_exp_internal,IMPORTANT
CONDITION,4,117,124,2,IMPORTANT
CONDITION,5,119,123,2,IMPORTANT
RETURN,2,126,126,2,IMPORTANT
```

Figure B.2. Example file for the ideal branch file

```
tau,Fr,t,zk_proof_systems/.../r1cs_ppzksnark.tcc,252
rhoA,Fr,rA,zk_proof_systems/.../r1cs_ppzksnark.tcc,300
rhoB,Fr,rB,zk_proof_systems/.../r1cs_ppzksnark.tcc,301
```

Figure B.3. Example file for the ideal value file

B.2.2 PrettyPrint Example

```
{static wnaf_window_table = {<std::_Vector_base<unsigned long, std::allocator<unsigned long> >> = {_M_impl = {<std::allocator<unsi
gned long>> = {<__gnu_cxx::new_allocator<unsigned long>> = {<No data fields>}, <No data fields>}, <std::_Vector_base<unsigned long
, std::allocator<unsigned long> >::_Vector_impl_data> = {_M_start = 0x5555556f1d90, _M_finish = 0x5555556f1db0, _M_end_of_storage
= 0x5555556f1db0}, <No data fields>}}, <No data fields>}, static fixed_base_exp_window_table = {<std::_Vector_base<unsigned long,
std::allocator<unsigned long> >> = {_M_impl = {<std::allocator<unsigned long>> = {<__gnu_cxx::new_allocator<unsigned long>> = {<No
 data fields>}, <No data fields>}, <std::_Vector_base<unsigned long, std::allocator<unsigned long> >::_Vector_impl_data> = {_M_sta
rt = 0x5555556f0810, _M_finish = 0x5555556f08c0, _M_end_of_storage = 0x5555556f0910}, <No data fields>}}, <No data fields>}, coord
 = {{<mie::local::addsubmul<mie::Fp, mie::local::comparable<mie::Fp, mie::local::hasNegative<mie::Fp, mie::local::inversible<mie::
Fp, mie::local::Empty<mie::Fp> > > > >> = {<mie::local::comparable<mie::Fp, mie::local::hasNegative<mie::Fp, mie::local::inversibl
e<mie::Fp, mie::local::Empty<mie::Fp> > > >> = {<mie::local::hasNegative<mie::Fp, mie::local::inversible<mie::Fp, mie::local::Empt
y<mie::Fp> > >> = {<mie::local::inversible<mie::Fp, mie::local::Empty<mie::Fp> >> = {<mie::local::Empty<mie::Fp>> = {<No data fiel
ds>}, <No data fields>}, <No data fields>}, <No data fields>}, <No data fields>}, v_ = {1523403791020821917, 754611498739239741,
738101653846473271 6, 1011752739694698287}}, {<mie::local::addsubmul<mie::Fp, mie::local::comparable<mie::Fp, mie::local::hasNegati
ve<mie::Fp, mie::local::inversible<mie::Fp, mie::local::Empty<mie::Fp> > > > >> = {<mie::local::comparable<mie::Fp, mie::local::ha
sNegative<mie::Fp, mie::local::inversible<mie::Fp, mie::local::Empty<mie::Fp> > > >> = {<mie::local::hasNegative<mie::Fp, mie::loc
al::inversible<mie::Fp, mie::local::Empty<mie::Fp> > >> = {<mie::local::inversible<mie::Fp, mie::local::Empty<mie::Fp> >> = {<mie:
:local::Empty<mie::Fp>> = {<No data fields>}, <No data fields>}, <No data fields>}, <No data fields>}, <No data fields>}, v_ = {12
014063508332092218, 1509222997478479483, 14762033076929465432, 2023505479389396574}}, {<mie::local::addsubmul<mie::Fp, mie::local:
:comparable<mie::Fp, mie::local::hasNegative<mie::Fp, mie::local::inversible<mie::Fp, mie::local::Empty<mie::Fp> > > > >> = {<mie:
:local::comparable<mie::Fp, mie::local::hasNegative<mie::Fp, mie::local::inversible<mie::Fp, mie::local::Empty<mie::Fp> > > >> = {
<mie::local::hasNegative<mie::Fp, mie::local::inversible<mie::Fp, mie::local::Empty<mie::Fp> > >> = {<mie::local::inversible<mie::
Fp, mie::local::Empty<mie::Fp> >> = {<mie::local::Empty<mie::Fp>> = {<No data fields>}, <No data fields>}, <No data fields>}, <No
data fields>}, <No data fields>}, v_ = {1523403791020821917, 754611498739239741, 738101653846473271 6, 1011752739694698287}}}}
```

(a) Content without PrettyPrint in `libsnark`

```
pairing::bls12_381::ec::g1::G1Affine {x: pairing::bls12_381::fq::Fq (pairing::bls12_381::fq::FqRepr ([10547801628701281408, 146510
83689357211131, 648164855365100991, 9048958586612376648, 8036176713143700390, 705443719916985888])), y: pairing::bls12_381::fq::Fq
 (pairing::bls12_381::fq::FqRepr ([8855955297991251329, 16069468485776047325, 12121465487721251295, 18418488638908299805, 16571651
195142601741, 1367670370568535616])), infinity: false}
```

(b) Content without PrettyPrint in `Bellman`

```
ark_ec::models::short_weierstrass::affine::Affine<ark_bls12_381::curves::g1::Config> {x: ark_ff::fields::models::fp::Fp<ark_ff::fi
elds::models::fp::montgomery_backend::MontBackend<ark_bls12_381::fields::fq::FqConfig, 6>, 6> (ark_ff::biginteger::BigInt<6> ([476
2646147170998592, 1278820419785028 3445, 6937814569682132977, 4601457130438782450, 2125856508034529 4, 7509243545107602 41]), core::m
arker::PhantomData<ark_ff::fields::models::fp::montgomery_backend::MontBackend<ark_bls12_381::fields::fq::FqConfig, 6>>), y: ark_f
f::fields::models::fp::Fp<ark_ff::fields::models::fp::montgomery_backend::MontBackend<ark_bls12_381::fields::fq::FqConfig, 6>, 6>
(ark_ff::biginteger::BigInt<6> ([9390487088130527322, 6856353769413480936, 18328620154857584441, 15200026662905747573, 74045709516
95852829, 1485648545527895280]), core::marker::PhantomData<ark_ff::fields::models::fp::montgomery_backend::MontBackend<ark_bls12_3
81::fields::fq::FqConfig, 6>>), infinity: false}
```

(c) Content without PrettyPrint in `arkworks`

```
(7808682176532415071898036103081779430057470046557408184205774905195797171 55, 9948895334884577511816681179891480858815618120919043
9551936421444370200750 12)
```

(d) Content with PrettyPrint in any library

Figure B.4. Content of variable type G1

B.3 Equations `comparison_gadget` Represents

$$
\begin{cases}
(w_{14} * (1 + (p-1) * w_{14})) \bmod p = 0 \\
(1024 + w_3 + 2 * w_4 + 4 * w_5 + 8 * w_6 + 16 * w_7 + 32 * w_8 + \\
\qquad 64 * w_9 + 128 * w_{10} + 256 * w_{11} + 512 * w_{12}) \bmod p = w_{13} \\
(w_3 * (1 + (p-1) * w_3)) \bmod p = 0 \\
(w_4 * (1 + (p-1) * w_4)) \bmod p = 0 \\
(w_5 * (1 + (p-1) * w_5)) \bmod p = 0 \\
(w_6 * (1 + (p-1) * w_6)) \bmod p = 0 \\
(w_7 * (1 + (p-1) * w_7)) \bmod p = 0 \\
(w_8 * (1 + (p-1) * w_8)) \bmod p = 0 \\
(w_9 * (1 + (p-1) * w_9)) \bmod p = 0 \\
(w_{10} * (1 + (p-1) * w_{10})) \bmod p = 0 \\
(w_{11} * (1 + (p-1) * w_{11})) \bmod p = 0 \\
(w_{12} * (1 + (p-1) * w_{12})) \bmod p = 0 \\
(1024 - 1 * w_1 + w_2) \bmod p = w_{13} \\
(w_{15} * (w_3 + w_4 + w_5 + w_6 + w_7 + w_8 + w_9 + w_{10} + w_{11} + \\
\qquad w_{12})) \bmod p = w_{14} \\
((1 + (p-1) * w_{14}) * (w_3 + w_4 + w_5 + w_6 + w_7 + w_8 + w_9 + \\
\qquad w_{10} + w_{11} + w_{12})) \bmod p = 0 \\
w_{14} \bmod p = 1
\end{cases}
$$